

ספר פרויקט במערכות אוטונומיות-כיתה י"ד

אקולייזר גרפי, נגן שמע



שם הסטודנט: דניאל אלוש

תעודת זהות: 215176850

שם מכללה: מקיף ה' דרכא אשקלון

סמל מוסד: 644450

שם מנחה: משה זזק

מספר שאלון: 714916

שם פרויקט: אקולייזר גרפי-graphic equalizer, נגן שמע

תוכן עניינים:

3	פתיחה
3	• מבוא
3	• תקציר
4	• הצהרת הבעיה
4	• מטרת הפרויקט
4	• הקשר בתוך מערכות משובצות מחשב
4	• היקף ומגבלות
5	• גישה אינטר-דיסציפלינרית
5	• שפת תכנות וסביבת עבודה
5	• שרטוט בלוקים
6	• שרטוט חיבור הרכיבים
7	רקע תיאורטי
7	• עיבוד אותות דיגיטליים-DSP
7	- תיאוריית הדגימה
7	- קוונטיזציה
7	• ייצוג שמע דיגיטלי
8	- מבנה פורמט קובץ WAV
8	- שמע מונופוני
8	• פילטרים דיגיטליים
8	- יסודות פילטרים(תגובת תדר, תגובת פאזה)
8	- פילטר FIR, IIR
9	- מבנה פילטר Biquad
9	- סוגי הפילטרים בשימוש
10	• עקרונות אקולייזר(equalizer) גרפי
10	- מאפייני אקולייזר גרפי
10	- עיצוב אקולייזר-פסי תדר
11	ישום חומרה
11	• ארכיטקטורת חומרה
11	• מפרט רכיבים
12	• חיווט של המערכת
13	• יישום פרוטוקולי תקשורת
13	• ממשק SPI
14	• ממשק I2S
14	• ממשק I2C
14	• המרת אנלוגית דיגיטלית-ADC
15	תכנון תוכנה
15	• תכנון ארכיטקטורת התוכנה-high-level block diagram
16	• אסטרטגיית תכנון התוכנה
16	- בחירת סביבת פיתוח-ESP-IDF

16	- שיקולי עיבוד בזמן אמת
18	ישום תוכנה
18	- לולאת בקרה ראשית
19	- תיאורי מודלים
19	- זרימת נתונים
19	• אלגוריתם ליבה-החלת הפילטרים וניהול השמע
19	- חישוב מקדמי הפילטרים
21	- יישום פילטר בזמן אמת-ליבת עיבוד DSP
22	- זרימת נתוני שמע
23	• לוגיקת עדכון ממשק משתמש
23	- קריאות רכיבי החומרה ומיפוי פרמטרים
24	- עדכוני צג
26	• ניתוח מורכבות אלגוריתם
26	- עומס חישובי של מסנני DSP
26	- אילוצי זמן אמת וניתוח תזמון
27	תיעוד הערכות ובדיקות המערכת
27	• תוכנית ונהלי בדיקה
27	• תוצאות וניתוח
27	- מדידת ביצועים
28	- הערכת איכות שמע
28	- אימות התאמות פס באקולייזר
28	• אתגרים שנתקלנו בהם ופתרונות
29	סיכום
29	• הישגי הפרויקט
29	• עבודה עתידית ושיפורים פוטנציאליים
29	- פלט סטריאו
29	- תמיכה בקלט שמע/משתמש נוספים
29	• סיכום

• מבוא

ספר זה מפרט את התכנון, היישום והערכה של מעבדה במערכות משובצות מחשב המתמקד בעיבוד שמע דיגיטלי. הפרויקט כרוך בפיתוח של אקולייזר (equalizer) גרפי ונגן קבצים בפורמט wav, המפותח על שבב מיקרו-בקר ESP32-S3 המתאפיין ביכולת עיבוד גבוהה ותמיכה בכלים המתאימים למשימות עיבוד שמע בזמן אמת. המערכת קוראת נתוני מנגינה/שיר מונופוני מקבצי wav המאוחסנים בכרטיס microSD, מפעילה פילטרים דיגיטליים הניתנים להגדרה על ידי המשתמש, ומוציאה את זרם השמע המעובד לרמקול דרך מגבר המחובר לבקר. פרויקט זה משלב עקרונות ומושגים מעולם עיבוד אותות דיגיטליים (DSP), תכנון חומרה משובצת מחשב ופיתוח תוכנה כדי ליצור התקן פונקציונלי למניפולציית שמע-תוך כדי שימוש מתאים במשאבים הנתונים.

• תקציר

פרויקט זה מתרכז בפיתוח ויישום של מערכת משובצת מחשב המיועדת לעיבוד שמע דיגיטלי בזמן אמת, המתפקדת כאקולייזר (equalizer) גרפי בעל 5 פסי תדרים ונגן קבצי wav מונופוני. המערכת, המונעת על ידי בקרת תדרים מפורטת, המנצלת את יכולות המיקרו-בקר ESP32-S3 כיחידת עיבוד מרכזית. המטרה העיקרית הייתה ליצור התקן עצמאי המסוגל לקרוא נתוני שמע מכרטיס זיכרון microSD, להחיל פילטרים דיגיטליים הבונים את האקולייזר-הניתנים לכוונון על ידי המשתמש, ולשלוח את אות השמע המותאם לרמקול, והכל תוך פעולה במסגרת מגבלות המשאבים של השבב איתו נשתמש.

ארכיטקטורת חומרה משלבת את המיקרו-בקר עם ציוד היקפי חיוני: מודל קורא כרטיסי microSD המחובר דרך SPI לקלט השמע הדיגיטלי, מגבר max98357a class D המקבל נתוני אודיו מעובדים דרך פרוטוקול I2S, שלושה פוטנציומטרים B10k המספקים קלט משתמש אנלוגי הנקרא דרך ערוצי ADC של השבב, וצג SSD1306 OLED המחובר דרך I2C למשוב חזותי למשתמש. מבחר רכיבים זה יוצר פלטפורמת חומרה מגובשת המסוגלת לתמוך בפונקציות הנדרשות של הקלט, העיבוד, והפלט והאינטראקציה עם המשתמש.

התוכנה שפותחה ב-C באמצעות Espressif IoT Development Framework-ESP-IDF גרסה 5.0.4, משתמשת בגישת ריבוי משימות המנוהלת על ידי FreeRTOS. המשימה הראשית מטפלת בצינור השמע - קריאת נתוני wav מכרטיס זיכרון, ניתוח כותרות וניהול מאגרי שמע. עיבוד השמע המרכזי כרוך בהפעלת חמישה מסנני IIR מסדר שני מדורגים במבנה Biquad, המיושמים באמצעות ספריית esp-dsp המותאמת, כדי לשנות את תוכן התדרים בהתאם להגדרות האקולייזר. באותה עת, משימה נפרדת מנהלת את ממשק המשתמש, קוראת באופן מחזורי ערכי פוטנציומטרים, ממפה את התוצאות לפרמטרים של עוצמת קול, בחירת פס התדר וההגבר באותו פס, ועדכון מצב המערכת והתצוגה החזותית. סנכרון קריטי באמצעות שיטת מניעה הדדית מבטיח גישה בטוחה למשתנים משותפים של האקולייזר בין ממשק המשתמש למשימות עיבוד שמע, מונעת פגיעה בנתונים ומבטיחה יישום יציב של הפילטרים.

למשך הפרויקט, הערכת הביצועים הייתה היבט מרכזי, בהשוואה לאסטרטגיות יישום תוכנה שונות. בדיקות הראו כי שימוש במסגרת ESP-IDF ובשימוש פונקציות DSP סיפק ביצועים טובים משמעותית בזמן אמת עבור משימת הפילטרים עתירת החישוב בהשוואה ליישומים ידיניים או שימוש במסגרת Arduino IDE, מה שהוכיח את עצמו כחיוני להשגת פלט שמע יציב וללא תקלות בקצב הדגימה המיועד שהצבנו-44.1 קילוהרץ. בדיקות פונקציונליות אישרו את הפעולה הנכונה של פסי התדרים של האקולייזר ובקורות המשתמש. הפרויקט התגבר בהצלחה על אתגרים הקשורים לאילוצי עיבוד וגישה בו-זמנית לנתונים, וכתוצאה מכך נוצר אב טיפוס המדגים את יישום התנהגות האקולייזר על השמע באמצעות עיבוד אותו דיגיטליים במערכת משובצת מחשב על ה-ESP32S3.

למרות שכרגע הינה מוגבלת להשמעת wav מונופוני, המערכת מספקת בסיס לשיפורים עתידיים כגון תמיכה בסטריאו, כניסות שמע נוספות או תכונות בקרה אלחוטיות.

• הצהרת הבעיה

בדרך כלל התקני שמע סטנדרטיים ומספקי שירותים דיגיטליים למוסיקה מציעים שליטה מוגבלת על תוכן התדר של השמע המתנגן דרכם, ומספקים בדרך כלל רק התאמות צליל בסיסיות או אקולייזרים גרפיים קבועים. לעיצוב שמע מדויק יותר, מתאים יותר לסיטואציה נתונה או פשוט בהתאם להעדפת המשתמש-כגון פיצוי על אקוסטיקה של המקומים או התאמת טווחי תדרים מסוימים להעדפה אישית, נדרש אקולייזר. יישום מערכת זו על פלטפורמה עצמאית מוגבלת מציבה אתגר עיבוד בזמן אמת וניהול היקפי יעיל. פרויקט זה עונה על הצורך להלן בפתרון אקולייזר שמע גרפי משובץ שבב מיקרו-בקר שניתן להגדירו.

• מטרת הפרויקט

- המטרות העיקריות של פרויקט זה הן:
- תכנון ויישום אקולייזר גרפי בעל 5 פסי תדר באמצעות פילטרים דיגיטליים על גבי בקר ESP32S3 בזמן אמת, המופעלים על זרם השמע לפני הפלט.
 - פיתוח גגן שמע המסוגל לקרוא ולשמוע קבצי wav מונופוניים מכרטיס microSD, תוך השפעת האקולייזר עליהם.
 - ליצור ממשק משתמש המאפשר כוונן עוצמות הקול, בחירת פסי תדר ושליטת ההגבר עליהם(הגברה או קיצוץ ביחידות של חוזק התדר, ב-dB), ולספק משוב חזותי וגרפי בנוגע לפרמטרים העכשוויים ופרטי האקולייזר החל על המוסיקה באמצעות צג.

• הקשר בתוך מערכות משובצות מחשב

פרויקט זה נמצא בין מחשוב משובץ מחשב לעיבוד שמע דיגיטלי בכך שמדגים את השימוש בבקר חיצוני לביצוע משימות עתירות חישוב כבד, מבינהם DSP בזמן אמת, אשר נתונות מגבלות לשבבים ייעודים מסוימים. מדגים את השילוב של הצידוד החומרתי ההיקפי הכרוך בסוגי תקשורות סטנדרטיים(SPI, I2S, I2C, ADC) הנפוצים בתכנון מערכות משובצות מחשב. יתר על כן, ישנה התמודדות עם אתגרים הכוללים אילוצי משאבים(כוח עיבוד, זכרון), דרישות פעולה בזמן אמת וממשק חומרה ברמה נמוכה יחסית-אתגרים אופייניים למערכות משובצות.

• היקף ומגבלות

- הפרויקט במצבו הנוכחי נמצא בהיקף הבא:
- ניגון קבצי wav pcm מונופוניים, עם עומק של 16 סיביות, קצב דגימה של 44.1kHz.
 - יישום של אקולייזר גרפי בעל 5 טווחי תדר, המשתמש בפילטרים biquad IIR המכונים peaking/notch וכ-shelving עבור כל פס טווח הנבחר המוגדר מראש. כמו כן התאמת ההגבר בטווח מוגבל המוגדר גם הוא מראש(9dB -> 9dB - בקפיות של 3dB).
 - פלט שמע מעובד עובר דרך פרוטוקול I2S למגבר max98357A לספיקר.
 - תצוגה של עוצמת קול נוכחית, פס תדר והגבר שנבחרו וייצוג גרפי בסיסי של עקומת האקולייזר על גבי צג.

כרגע נתון במגבלות הבאות:

- המערכת תומכת בקבצי wav העונים על ההסבר הספציפי שתואר, קובץ שאינו נמצא בפורמט wav, בעל עומק 16 סיביות וקצב דגימה השונה מ-44.1kHz לא יתנגן כשורה. כל קודקים אחרים של שמע לא מטופלים.
- ממשק המשתמש מתבסס על 3 פוטנציומטרים, אין כניסות כפתורים או מערכת תפריט מתקדם-כניסת השמע מוגבלת לקבצים מאוחסנים בכרטיס microSD.

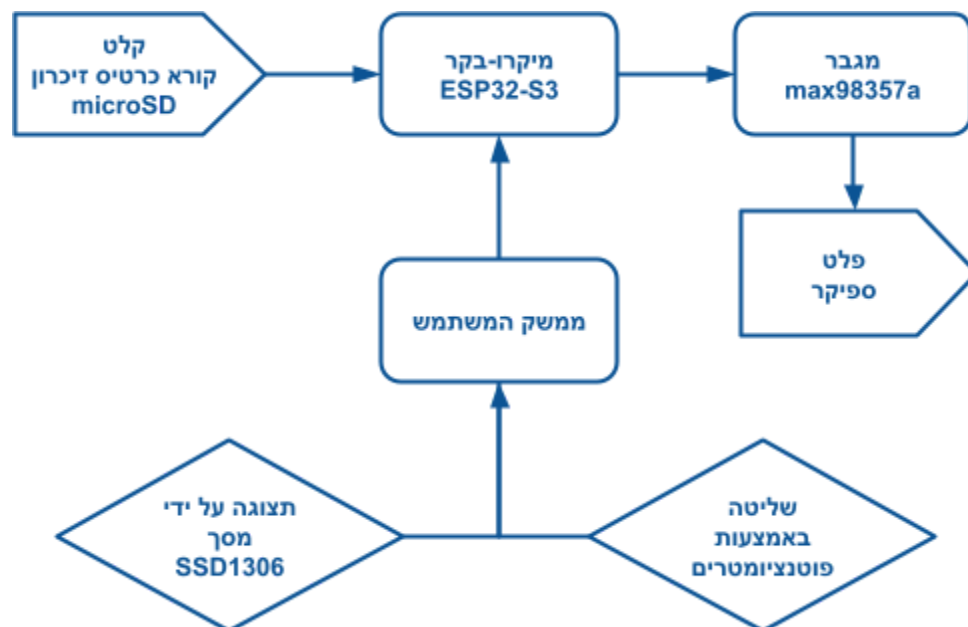
● גישה אינטר-דיסציפלינרית

- הצלחת פרויקט זה כרוכה בשילוב של דיסציפלינות שונות, שיחד עונות על מטרת המערכת, וכל אחת מהן תורמת בעקרונותיה ומומחיות ייחודיות לאותו שדה מחקר, המתואר להלן:
- **תכנות** בשפת C באמצעות סביבת העבודה ESP-IDF, מושגי תכנות בזמן אמת (שימוש בריצה אסינכרונית באמצעות FreeRTOS ו-mutual exclusions), יישום ושימוש בדרייברים המסופקים עם הבקר, ניהול זיכרון ותכנון אלגוריתמים לעיבוד שמע ובקרת ממשק למשתמש.
- **חומרה משובצת**-בחירת רכיבים (מיקרו-בקר, מגבר, תצוגה ופוטנציומטרים), התממשקות בפרוטוקולי תקשורת (SPI, I2S, I2C) וקריאות כניסות אנלוגיות (ADC). כמו כן התאמת המעגל האלקטרוני תוך התחשבות בדרישות הפרויקט.
- **עיבוד אותות דיגיטליים**-עקרונות שיטות בתחום זה, הבנת תיאוריית הפילטרים (IIR, Biquad), תגובת תדר (frequency response) ועקרונות ומבנה שמע דיגיטלי.
- **עיצוב מכני-תכנון** והרכבה של התוצר הסופי באמצעות חלקים מודפסים בתלת מימד שדורשים שיקולים כגון קשיחות, דיוק והרכבה פיזית הינם בעלי חשיבות לשמירת האסתטיקה של המערכת בהשגת נוחות המשתמש.

● שפת תכנות וסביבת עבודה

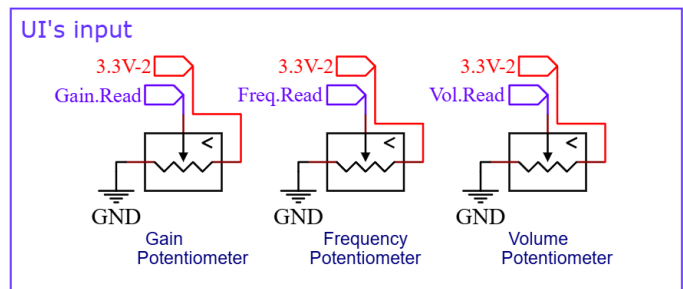
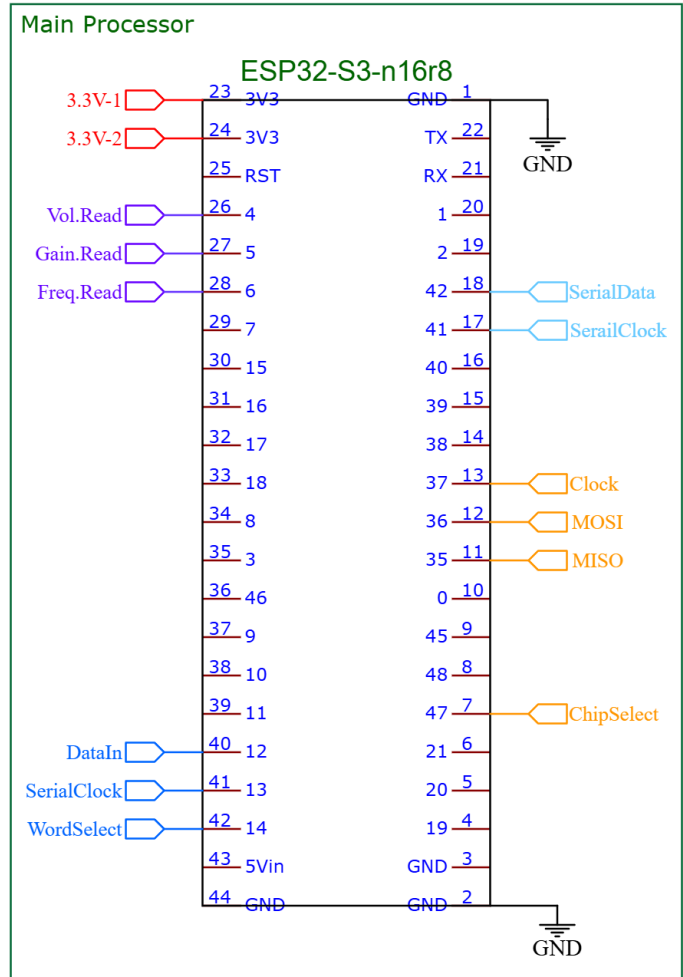
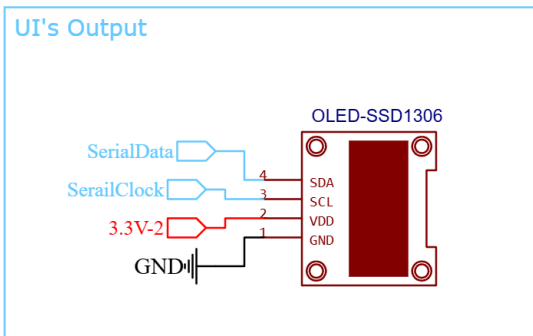
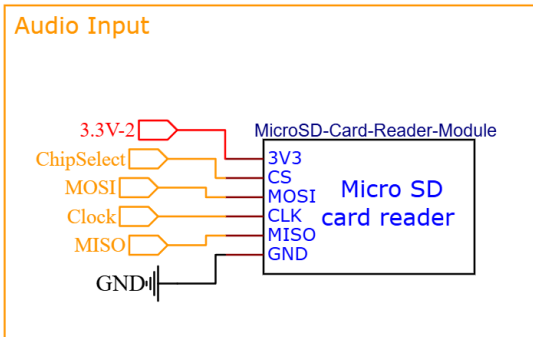
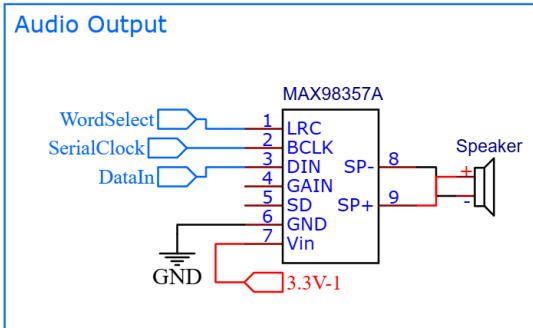
התכנות (וה-firmware) עבור הבקר ESP32-S3 פותחה באמצעות שפת התכנות C. סביבת הפיתוח בה נעשה שימוש הייתה ESP-IDF - Espressif IoT development framework גרסה 5.4.1, המספקת דרייברים ספריות הכרחיות, ליבת FreeRTOS וכלי בנייה הדרושים לפיתוח יישומים על פלטפורמת ESP32.

● שרטוט בלוקים



• שרטוט חיבור הרכיבים

סכמת חיווט וחיבור הרכיבים אחד לשני מתוארים להלן:



רקע תיאורטי

מקטע זה בספר יכלול את הרקע להבנת העקרונות עליהם מבוסס אקולייזר (equalizer) והנגן שמע. יכלול את מושגי הבסיס עבור עיבוד אותות דיגיטליים, ייצוג שמע וקבצי שמע, שיטות פילטרים דיגיטליים ורעיונות וכללים ספציפיים של אקולייזר גרפי בקשר לפרויקט זה.

• עיבוד אותות דיגיטליים (DSP)

תיאוריית DSP עוסקת בייצוג של אותות רציפים מהעולם האמיתי (גלים מכניים) כרצפים של מספרים (אותות דיגיטליים) ועיבודם. מכילה פונקציות ושיטות רבות ליישום פעולות מטריציניות באופן יעיל, אופטימיזציה מרבית במיוחד בשבבים שתחומים במשאבים מוגבלים. נתמקד בזאת המרת ועיבוד שמע לפורמט דיגיטלי כרוך בשני שלבים עיקריים: דגימה וקוונטיזציה.

- תיאוריית הדגימה

אותות שמע במקורם הם רצפים בזמן ובאמפליטודה, דגימה היא תהליך של המרת אות בזמן רציף לאות בזמן בדיד על ידי ביצוע מדידות (דגימות) במרווחי זמן קבועים באמצעות כלים מעולם עיבוד השמע-Fourier Transform. הקצב שבו נלקחות דגימות אלה הוא תדר הדגימה או קצב הדגימה (לשימוש, נסמן כ- F_s) נמדד ביחידות מידה של הרץ (Hz)-במקרה שלנו הוא $F_s = 44100$ [Hz].
ע"פ התיאוריה (Nyquist Shannon sampling theorem) האומרת כי תדר הדגימה חייב להיות גדול מפי 2 מרכיב התדר המקסימלי הקיים באות המקורי (F_{max}), דבר שימנע בעיה (Aliasing): אם קצב הדגימה נמוך מדי ($F_s \leq 2 \cdot F_{max}$) תדרים מעל חצי קצב הדגימה "יתקפלו חזרה" בצורה מודולרית לטווח תדרים נמוכים, ויופיעו תדרים שגויים שבמקור לא קיימים. עיוות זה נקרא aliasing הינו בלתי הפיך. עבור שמע טווח השמיעה האנושי הוא עד כ-20kHz ולפיכך נדרש קצב דגימה של מעל 40kHz.

- קוונטיזציה

לאחר שהדגימה תופסת את האות בנקודות זמן דיסקרטיות, האמפליטודה של כל דגימה עדיין נשארת ערך רציף, מערכות דיגיטליות יכולות לאחסן רק מספרים מדויקים סופיים. קוונטיזציה היא תהליך של קירוב כל אמפליטודה עם ערך מקבוצה סופית של קבוצת רמות דיסקרטיות. מס' רמות זה נקבע על ידי עומק הסיביות של ממיר אנלוגי לדיגיטלי או פורמט שמע בצורה דיגיטלית, כך שעומק גדול יותר מאפשר ליותר רמות ולכן קירוב מדויק יותר לאמפליטודה המקורית. לדוג' בפרויקט זה נעשו ניסויים על עומקי סיבית שונות-התבססו על 16 סיביות, משמע ישנן $2^{16} = 65536$ רמות שונות.
לכל ערך מומר שכזה ישנה שגיאה שמכניסה רעש לאות דיגיטלי שמכונה 'רעש'. איכות הקוונטיזציה נמדדת בין היתר על ידי יחס רעש לאות-אשר משווים את עוצמת האות הרצוי לעוצמת הרעש. עבור 16 סיביות על ידי מדידה זאת רעש הקוונטיזציה יהיה נמוך מאוד יחסית לרמת האות המקסימלית, במידה ונלקח 8 סיביות יחס זה יעלה וכך מורגש יותר הרעש, דבר שנרצה להימנע בדרך כלל בהשמעת המוזיקה.

• ייצוג שמע דיגיטלי

לאחר דגימה וקוונטיזציה, אחסון ובניית נתוני שמע תחומות בכללים של מבנה פורמטים שונים של קבצי שמע דיגיטליים, במקרה שלנו נלקח פורמט קובץ wav, אשר מספק דגימות שמע גולמיות ומאפייני שמע דיגיטלי בצד חומרת של התקני שמע;

- מבנה פורמט קובץ WAV

דרך סטנדרטית לאחסון דגימות השמע יחד עם נתונים (metadata) חיוניים כדי שמערכות השמעה יפרשו את הנתונים בצורה נכונה היא להשתמש בקובץ פורמט wav; פורמט זה הינו נפוץ לאחסון ושמירת שמע לא דחוס, המשתמש במבנה חילופי משאבים (RIFF) המארגן נתונים לבלוקים (chunks) מתויגים של מידע. מבנה זה כולל: את התג RIFF עם גודל הקובץ בפורמט ('WAVE') ואת פורמט השמע שנשתמש, פורמט שמע (PCM), מס' ערוצים, קצב דגימה, עומק סיביות ועוד. המידע של השמע עצמו ימצא ב-chunks שמכיל את דגימות אודיו בפועל. לצורך הסבר תוכן קובץ ה-wav, שיטת PCM-pulse code modulation הינה השיטה הנפוצה ביותר לייצוג דיגיטלי של אותות אנלוגיים, הפלט הישיר של תהליך הדגימה. ב-PCM ליניארי, הערך המספרי מוקצה לכל דגימה הוא ביחס ישר לאמפליטודת האות באותו רגע דגימה. לאחר קוונטיזציה של כל דגימה, רצף המספרים יוצר את זרם נתוני השמע של PCM. קובץ ה-wave איתו נעסוק מכיל נתוני PCM לא דחוסים.

- שמע מונופוני

אודיו מונופוני (מונו) משתמש בערוץ יחיד כדי לייצג את הצליל, כל מידע השמע משולב בערוץ אחד זה, ויוצר תמונת שמע הממוקמת בנקודה אחת במרחב (אם נמצא המערכת סטריאו אז יהיה ממורכז בין הרמקולים)- זאת בניגוד לשמע סטריאופוני (סטריאו) המשתמש בשני ערוצים, ימין ושמאל, כדי ליצור את תחושת השמע המרחבי כיווני. בפרויקט זה השימוש הוא בשמע מונו מפאת המגבלה של משאבים מוגבלים הנתונים לנו, זאת מכיוון בקובץ שמע מונופוני יש צורך לקרוא ולעבד חצי מכמות הנתונים המאוחסנים בזיכרון, בהשוואה לסטריאו באותו קצב דגימה ועומק סיביות. משמע יש להחיל פילטרים רק על זרם דגימות אחד ובכך לתחזק רק קו נתונים I2S אחד וערוץ/רמקול אחד.

• פילטרים דיגיטליים

פילטרים דיגיטליים הם אלגוריתמים, מעין טרנספורמציה הפועלת על אותות דיגיטליים כדי לשנות את תוכן התדר שלהם והם חיוניים לאקולייזר שנממש לעיל.

- יסודות פילטרים (תגובת תדר, תגובת פאזה)

פילטר דיגיטלי משמש להגברה/לחתיכה באופן סלקטיבי טווחי תדרים ספציפיים באות השמע, מבין היתר להשיג את האיזון הרצוי (אקואליזציה). לוקח רצף קלט $x[n]$ (מדגימות השמע המקורית) ומייצר את רצף הפלט $y[n]$ (הדגימות המעובדות), כאשר n מייצג את מדד הזמן הבדיד-מס' הדגימה. דרך אחת לתאר ולהמחיש את התנהגות פילטר הוא דרך תגובת תדר (frequency response), אשר מתאר איך פילטר משפיע על אות סינוסואידלי בתדרים שונים ויש לו שני מרכיבים: $|H(f)|$ המייצג את ההגברה/הנחתה המופעלת בכל תדר, ומוצג ביחידות מידה של דציבלים (dB). תגובת הפאזה (phase response) מוסמן כ- $\angle H(f)$ המייצג את עיכוב הזמן, הסתה שמוצג על ידי הפילטר בכל תדר-למרות שהוא קריטי ביישומים מסוימים, עיוות זה יכול לעזור לאיזון בסיס של תגובת הפילטר על זרימת שמע.

- פילטר FIR, IIR

ישנם דפוסים שונים של פילטרים המתחלקים לשתי קבוצות, פילטר מסוג finite impulse response, פילטר נאיבי אשר הפלט שלהם תלוי רק בקלטים הנוכחיים ותגובתם הינה סופית-בניגוד אליו יש פילטר מסוג infinite impulse response אשר שהפלט שלו $y[n]$ תלוי לא רק בדגימות נוכחיות ($x[n]$, $x[n-1]$...) אלא גם בדגימות פלט קודמות ($y[n]$, $y[n-1]$...) תלות רקורסיבית זו היא מעין משוב, פשוטו כמשמעות שמו-תגובת פילטר לקלט יחיד יכול תיאורטי להימשך לנצח.

לכל פילטר מסוג IIR ישנם מקדמים לפיו מתאפיין, כאשר קבוצת המקדמים $b_1, b_2, \dots$ הינם מקדמים המוחלים על האותות של הקלט שחוזרים כמשוב אל עיבוד האות המקורי-נקראים מקדמי משוב, וקבוצת המקדמים $a_1, a_2, \dots$ הינם מקדמים המוחלים על זרם אות השמע שנכנס כקלט אל הפילטר-נקראים מקדמי הזנה. כל מקדם כזה נצמד אל דגימה שונה בציר הזמן ביחס לדגימה הנתונה, ונראה את שימושם בהמשך.

שימוש ב-IIR שנבחר בפרויקט זה למטרת השגת האקולייזר טמון בכך שניתן להשיג תגובת תדר המגביה/מקצצת טווח תדרים רצוי עם פילטר מסדר נמוך בהרבה לעומת פילטר FIR-דבר קריטי במערכת בעלת משאבים מוגבלים. לפילטר זה ישנו מס' מבנים, ארכיטקטורות שונות לשימושים שונים, במקרה שלנו נממש את מבנה פילטר biquad.

- מבנה פילטר Biquad

מבנה הפילטר הוא מקטע פילטר IIR מסדר שני, ונקרא כך מכיוון שפונקציית ההשפעה שלו היא יחס של שני פולינומים ריבועיים. מציע דרך יציב ומודולרית יחסית לבניית מס' סוגי פילטרים בעלי התנהגות שונה. ישנם שתי דרכים להסתכל על אות שמע העוברת דרך פילטר זה:

- צורה ראשונה הינה תיאור היחס בין הקלט והפלט בציר הזמן, נקראת משוואת הפרשים (Difference equation in direct form I):

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]$$

- כאשר הפלט והקלט מאופיינים אחד בעזרת השני וכך נוכל לקבל את רצף הפלט עבור דגימה מסוימת. צורה שנייה הינה תיאור התנהגות פילטר בציר התדר, ונקרא פונקציית העברה (Transfer function, Z-domain) כאשר z הינו אמפליטודת התדר:

$$H(z) = Y(z)/X(z)$$

$$Y(z) = b_0 + b_1 z^{-1} + b_2 z^{-2}$$

$$X(z) = 1 + a_1 z^{-1} + a_2 z^{-2}$$

כדי לחשב את הקלט נצטרך לאחסן שתי דגימות הקלט הקודמות את שתי דגימות הפלט הקודמות, ואלו יהיו משתני המצב של הפילטר. היישום יעשה על פי השימוש והצורך לאותו פלט רצוי, ובמקרה שלנו כל דגימה שכזאת תכול חמישה פילטרים שונים במבנה זה שלאחר כל ההרכבה הזה יוצר פלט של פילטר biquad אחד הופך לקלט לבא אחריו.

- סוגי הפילטרים בשימוש

אקולייזר דורש התנהגויות של פילטר שונות על מנת לעצב ולשלט בחלקים שונים של ספקטרום השמע בתחום התדרים, במקרה שלנו נעשה שימוש בשני סוגי פילטרים המבוססים על ארכיטקטורת פילטר Biquad מסדר שני.

- פילטר שיא/חריץ (peaking/notch): פילטר המגביר (שיא) או חותך (חריץ) טווח ספציפי של תדרים הממוקמים סביב תדר מרכזי רצוי. רוחב הפס (Band width) או פקטור האיכות (Q) שלו קובעים עד כמה התנהגות הפילטר באותו תדר יהיה מפוזר ורחב, או צר ומרוכז בטווח תדרים סביב המרכז. סוג זה משומש על פסי התדר הממוקמים בפסי הביניים של האקולייזר.
- פילטר מדף (shelf): פילטר המגביר או חותך מתחת לפינה מוגדרת בתדר מסוים- f_c , עם הגברה/חיתוך של צד אחד מאותו תדר, ישנם שני סוגים:
 1. פילטר מדף גבוה-מגביר/חותך את כל התדרים שמעל f_c , אותו ניישם לבקרת תדרים גבוהים.
 2. פילטר מדף נמוך-מגביר/חותך את כל התדרים שמתחת f_c , אותו ניישם לבקרת תדרים נמוכים.

חמשת המקדמים (b_0, b_1, b_2, a_1, a_2) של כל פילטר מחושבים על סמך סוג הפילטר הרצוי, הפרמטרים הרצויים (תדר אמצע שבו יוחל הפילטר, ההגבר, ורוחב טווח התדרים מושפעים) וקצב הדגימה של המערכת. קיימות נוסחאות סטנדרטיות ליושבים אלה, ימצאו בנספחים.

• עקרונות אקולייזר (equalizer) גרפי

בעוד שהפרויקט מחקה את אופן הפעולה של אקולייזר גרפי, ישנם מאפיינים משולבים עם סוג פרמטרי-מכיוון שהתצוגה החזותית דומה לממשק הגרסה הגרפית, הפרמטרים אותם פסי תדר מתנהגים כאקולייזר פרמטרי; אקואליזציה-פעולת האקולייזר, הוא תהליך של התאמת עוצמת הקול של תחומי תדרים שונים בתוך זרם. המרכיבים, הציוד ומעגלים המשמשים להשגת פעולה זאת, נקראים אקולייזר (equalizer). הדבר נותן אספקת שליטה גמישה על ספקטרום השמע מעבר לבס/טרבל (bass, treble) פשוטים או מחווי אקולייזר גרפי קבוע.

- מאפייני אקולייזר גרפי

ראשית, הצורך בו הוא לספק דרך אינטואיטיבית ויזואלית להתאמת התדרים על פני מספר פסי תדרים קבועים בו זמנית. אופן הפעולה שלו הוא חילוק ספקטרום השמע למספר פסים רציפים, שכל אחד מהם מרוכז בתדר מרכזי קבוע (F_c), משתמש בפילטר שהזכרנו לעיל, שרוחב הפס שלו קבועים גם כן. הפרמטר המתכוון היחיד עבור על פס הוא ההגבר (נמדד בדציבלים, dB), שבדרך כלל נשלט על ידי מחוון אנכי. מיקום המחוונים הללו מספקים מספק גרף חזרתי כס של עקומת תגובה התרד שמתקבל-ומכאן השם 'גרפי'. הסוג הפרמטרי דורש הבנה על השפעת רוחבי פס בהגברים שונים והיחס לשינוי בתדרים אחרים שיש להם קשר ביניהם-הגרפי לעומת זאת, מאפשר כיוון עוצמות יותר אינטואיטיבי למשתמש בו.

- עיצוב אקולייזר-פסי תדר

תדרי המרכז הקבועים מרווחים בדרך כלל באופן לוגריתמי על פני ספקטרום התדרים בשמע, כדי להתאים את ההרגשה לתפיסה האנושית של גובה התדר והצליל. לדוג' במקרה שלנו נבחרו תדרי האמצע 400, 1000, 2500, 6300, 16000 הרץ. הממשק הקלאסי של האקולייזר הגרפי מרוכב מחוונים אנכיים, אחד לכל פס תדר-הזזת מחוון למעלה מגבירה את פס התדר, בעוד שהזזתו למטה חותכת אותו. כפי שהוזכר, כל פס מקבל את קבוצת הפילטרים שלו האחראית לשינוי בהגבר שלו-המחוון המתאים לו בגרף, ולאחר מכן כל פלט שעבר את השינוי מסוכם יחד עם האות המקורי כדי לייצר את השמע הסופי המותאם לאקולייזר. לפיכך, תגובת התדר הכוללת של האקולייזר, היא סכום התגובות של כל פסי הפילטרים הבודדים בהגדרות ההגבר המתאימות להם.

ישום חומרה

• ארכיטקטורת חומרה

שכבה עליונה: ממשק משתמש גרפי ופיזי

- שלושה פוטנציומטרים השולטים על שלושה פרמטרים-עוצמת קול, פס תדר והגבר.
- אינדיקציה גרפית על צג המורה על ערכי הפרמטרים ומעמד האקולייזר חזותית.
- כרטיס אחסון זיכרון micro SD המכיל את קבצי השמע שיתנגנו.
- מתקשרים עם הבקר דרך הממשקים: SPI, I2C, ADC

שכבה אמצעית: עיבוד נתונים בבקר ESP32-S3

- קורא את קבצי השמע ומאחסן ב-buffer ומקבל את נתוני בחירת המשתמש.
- מעבד את זרם השמע בהתאם.
- מוציא את אות הפלט אל המגבר באמצעות חיווט ישיר בממשק I2S.

שכבה תחתונה: פלט השמע

- מגבר שמע מונופוני המקבל את קלט מהבקר.
- מעביר הלאה אל הרמקול וכך מנגן את המוזיקה.

• מפרט רכיבים

במערכת זאת נעשית שימוש ברכיבים להלן:

שבב מיקרו-בקר: ESP32-S3 n16r8 wroom 1

- תפקיד: יחידת העיבוד המרכזית (CPU) ובקר של המערכת כולה. מפעילה את ה-firmware שקוראת את נתוני השמע, מחשבת חישובי DSP, מטפלת בקלט המשתמש, מעדכנת את התצוגה ומנהלת את כל התקשורת הפיזית.
- מאפיינים:
 - מעבד 32bit xtensa lx7 בעל שתי ליבות
 - עובד בקצב מרבי של 240MHz
 - אחסון של 16MB
 - זיכרון נדיף PSRAM של 8MB

כניסת שמע: מודל קורא כרטיסי SD

- תפקיד: מהווה אחסון לא נדיף של קבצי שמע ונמצא על מצע ממשק SPI
- מאפיינים:
 - מאפשר לכרטיסי זכרון בעלי מערכות FAT32 להתנהג כאחסון נוסף לבקר.

מגבר פלט שמע: MAX98357A

- תפקיד: מגביר את אות השמע המעובד לרמה מספקת להנעת רמקול.
- מאפיינים:
 - פעולה מסוג class D שמציעה יעילות גבוהה (עד כ-93%), וצורך אנרגיה נמוכה.
 - מקבל קלט שמע דיגיטלי ישירות דרך פרוטוקול I2S
 - מספק הספק פלט טוב-לדוג' 3.2 וואט עבור רמקול 4 אוהם באספקת 5 וולט.
 - הגבר שניתן לבחירה דרך פין GAIN, בין 3 ל-15 דציבלים.
 - פועל מאספקת חשמל יחידה בין 2.5 וולט עד 5.5 וולט.

מפרט רמקול:

- תפקיד: מתמיר את אות השמע החשמלי המוגבר לגלי קול נשמעים, פלט הסופי של המערכת.
- מאפיינים:
מודל LHS-D6236T LG.
התנגדות של 4 אוהם, עד 40 וואט.
מתאים לשימוש עם מגבר השמע הנתון לנו.
טרמינל חיבור סטנדרטי של חיבור אדום/שחור.

בקורות משתמש: פוטנציומטרים B10K

- תפקיד: לספק ממשק אנלוגי למשתמש כדי לכוון את פרמטרי המערכת: עוצמת הקול הכוללת, פס התדרים של האקולייזר הנבחר, וההגבר המופעל על הפס הזה.
- מאפיינים:
יחס ליניארי בין זווית סיבוב הציר לשינוי ההתנגדות.
התנגדות כוללת בין שני הדקים קבועים חיצוניים הוא 10 קילו-אוהם.
מהווה כמחלק מתח בין VCC ל-GND אשר נקראים ממנו קריאות אנלוגיות לבקר.

תצוגה: צג OLED SSD1306

- תפקיד: מספק משוב חזותי למשתמש המציג את המצב הנוכחי של האקולייזר.
 - מאפיינים:
טכנולוגית OLED שמספקת ניגודיות טובה ולאחר שאין תאורה אחורית, הדבר מוביל לצריכת חשמל נמוכה.
רזולוציה של 128x64 המשתרע על מסך בגודל 0.96"
- ממשק תקשורת I2C

• חיווט של המערכת

להלן חיווט הרכיבים עם המיקרו-בקר, ניתן לראות את סכמת החיבורים בעמוד 6 של הספר.

ESP32-S3 ↔ SD card module

ESP32-S3	47	36	35	37	3.3V	GND
SD mod	CS	MOSI	MISO	CLK	3.3V	GND

ESP32-S3 ↔ MAX98357a

ESP32-S3	14	13	12	N.C.	N.C.	3.3V	GND
MAX98357a	LRC	BCLK	DIN	GAIN	SD	3.3V	GND

ESP32-S3 ↔ SSD1306

ESP32-S3	41	42	3.3V	GND
SSD1306	SCK	SDA	VDD	GND

SP32-S3 ↔ Potentiometers

ESP32-S3	4	3.3V	GND
Volume B10K potentiometer	mid-pin	right-pin	left-pin
ESP32	5	3.3V	GND
Gain B10K potentiometer	mid-pin	right-pin	left-pin
ESP32	6	3.3V	GND
Frequency B10K potentiometer	mid-pin	right-pin	left-pin

• יישום פרוטוקולי תקשורת

על מנת לתקשר עם הרכיבי השונים המרכיבים בסופו של דבר את האקולייזר, נצטרך להשתמש בממשקים הנתמכים על ידי הרכיבים, ופרוטוקולי תקשורת להלן.

• ממשק SPI

- **פרטי פרוטוקול: SPI** (ממשק היקפי טורי-serial peripheral interface) הוא פרוטוקול טורי סינכרוני, דו צדדי מלא, בין מאסטר לעבד (master and slave). משתמש בארבעה קווי לוגיקה עיקריים:



1. שעון טורי-SCLK
נוצר על ידי המאסטר (ESP32) כדי לסנכרן את העברת הנתונים.
2. יציאת מאסטר, כניסת עבר-MOSI
קו נתונים ממאסטר לעבד.
3. כניסת מאסטר, יציאת עבר-MISO
קו נתונים עבד למאסטר.
4. בחירת עבד-CS

נוצר על ידי המאסטר, כדי לבחור עם איזה התקן עבד לתקשר כאשר קיימים מספר עבדים.

- **זרימת הנתונים:** הבקר יוזם את התקשורת על ידי משיכת ה-CS של כרטיס הזיכרון. שולח בייט של פקודה/כתובת באופן סדרתי לכרטיס ה-SD (עבד) דרך קו MOSI, מסונכרנים על ידי אות SCLK. כרטיס SD מעבד את הפקודה.

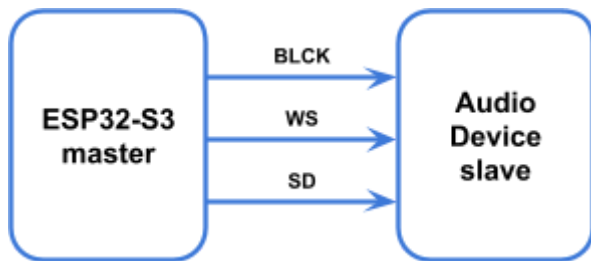
אם מתבקשים נתונים (קריאה), כרטיס ה-SD שולח בייט של נתונים בחזרה לבקר דרך קו MISO, ואם הבקר כותב, שולח נתונים דרך MOSI. התקשורת מסתיימת כאשר הבקר מעלה את קו CS לגובה. זרימה זאת משמשת לאתחול כרטיס, ניווט במערכת הקבצים (FAT32) וקריאת בלוקים של נתונים מקובץ wav נבחר.

יש לציין, כי המאסטר, המיקרו-בקר, הוא זה שבוחר את העבד הספציפי ואת השעון הסריאלי, ומנהל שני ערוצי תקשורת ביניהם: MOSI- עבד → בקר ו-MISO- בקר → עבד.

● ממשק I2S

- **פרטי פרוטוקול:** I2S-inter IC Sound הוא פרוטוקול טורי שתוכנן במיוחד להעברת נתוני שמע דיגיטליים. משתמש בדרך כלל בשלושה קווים עיקריים:

1. שעות-CLK bit BCLK
אות שעון לסנכרון ביטים של נתונים. כל פולס (pulse) מתאים לביט נתונים אחד. השליחה שלו הוא $sample\ rate * bits\ per\ channel * num\ channels$
2. בחירת ערוץ-WS, word select / LRC, left right clock
מציין את הנתונים של הערוץ (ימין או שמאל) המשודרים כעת, בדרך כלל עובר לפי קצב הדגימה.
3. נתונים סריאלי-SD, serial data
נושא את נתוני השמע בפועל, תוך ריבוב נתוני ערוץ ימין ושמאל בהתבסס על אות בחירת הערוץ.



- **זרימת הנתונים:** הבקר מייצר את אותות השעון ובחירות הערוץ. השעון רץ בתדר קבוע של $sample\ rate * bits\ per\ channel * num\ channels$
בעוד שבחירת הערוץ רץ בתדר של $sample\ rate$
הבקר שולח דגימות שמע של 16 ביט באופן סדרתי אל ה-DIN שעל המגבר, מכיוון שהפלט הוא מונו, אותם נתונים נשלחים רק עבור ערוץ אחד.

● ממשק I2C

- **פרטי פרוטוקול:** I2C-inter integrated circuit הוא פרוטוקול סינכרוני, חצי דופלקס המשתמש בשני קווים בלבד.

1. נתונים סריאליים-SDA, serial data
קו דו כיווני להעברת נתונים.
2. שעות סריאלי-SCL, serial clock
שעון הנוצר על ידי הבקר כדי לסנכרן את העברת הנתונים.



- **זרימת הנתונים:** רצף התקשורת בין שני צדדים מתבצע באופן הבא: הבקר מושך SDA נמוך בעוד ש-SCL גבוה, ושולח כתובת יעד של 7 סיביות וסיבית המורה על קריאה/כתיבה, היעד מושך SDA נמוך במשך מחזור שעון אחד כדי לאשר קבלת כתובת, אם אף יעד אינו מגיב אין אישור. הבקר/היעד שולחים בתים של 8 סיביות והמקלט מאשר כל בייט. כאשר הבקר מושך SDA גבוה בעוד ש-SCL נמוך, התקשורת מסתיימת.

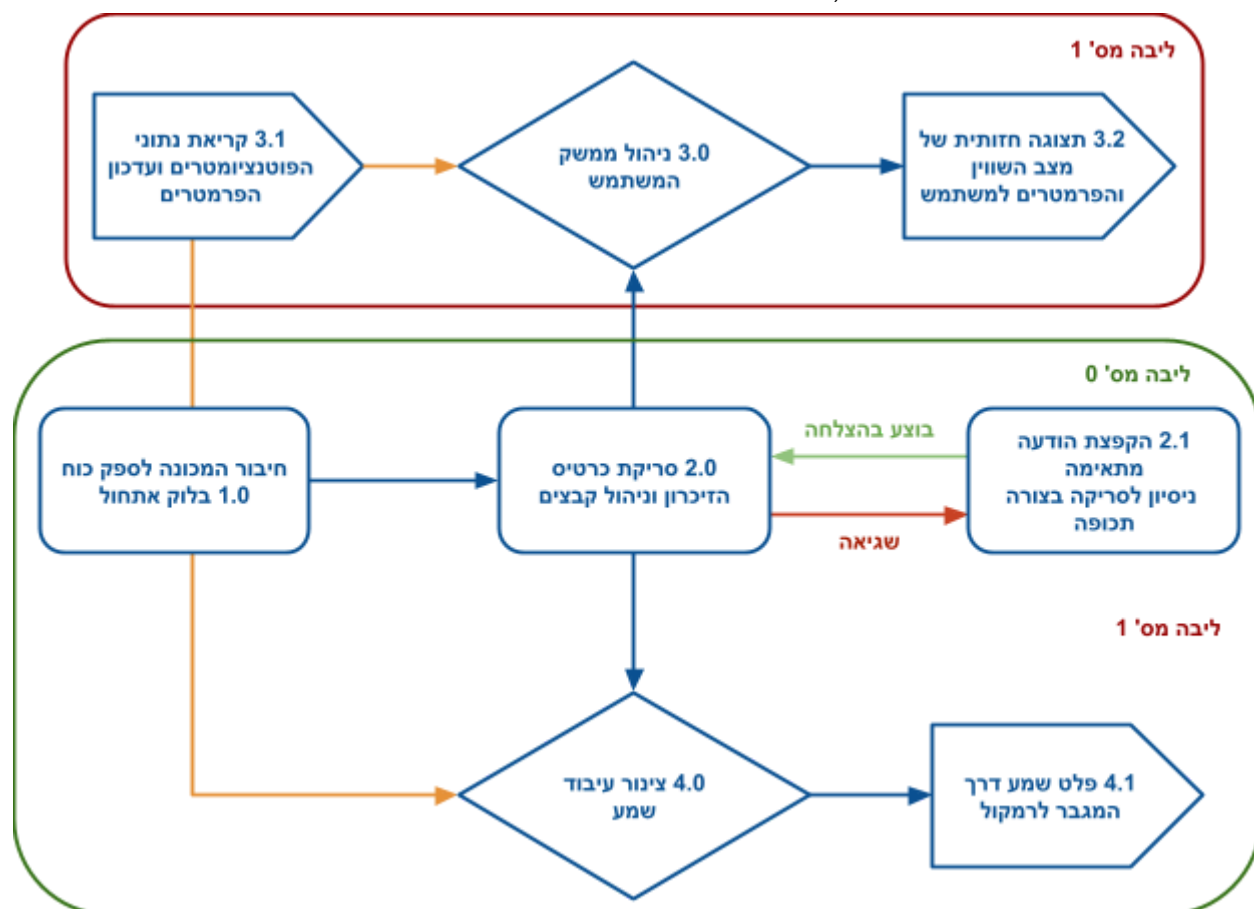
● המרת אנלוגית דיגיטלית-ADC

- **פרטים:** המרה זאת אינה פרוטוקול תקשורת בין התקנים כמו הממשקים האחרים, אלא פונקציה היקפית בתוך הבקר אשר ממירה מתח אנלוגי חיצוני לייצוג דיגיטלי.

תכנון תוכנה

• תכנון ארכיטקטורת התוכנה (high-level block diagram)

ארכיטקטורת התוכנה תהיה מתוכננת ובנויה סביב מספר בלוקים בעלי פונקציונליות שונה, אשר פועלים ביניהם כדי להשיג את מטרת המערכת, נמחיש את המערכת הצורה הבאה:



להלן הסבר על תרשים הזרימה המעשית של התוכנה:

1. בלוק אתחול: בעת אספקת החשמל למערכת, הבלוק אחראי על הגדרת חומרת השבב בהפעלתו. זה כולל הגדרות חיווט עבור הפונקציונליות המיועדת להן (ממשקי תקשורת-SPI, I2C, I2S, ADC), אתחול מנהלי התקנים היקפיים וסטנדרטיים מובנים, הרכבת מערכת הקבצים של כרטיס ה-SD, אתחול מבני נתונים מרכזים ומניעות הדדיות (mutex) ומשימות.
2. בלוק ניהול כרטיס SD וקבצים: מטפל בכל האינטראקציות עם כרטיס הזיכרון זה כרוך בחיפוש קבצי wav, פתיחת הקובץ שנבחר, ניתוח כותרת הקובץ כדי לחלץ את פרמטרי שמע וקריאת תחילת שמע ברצף מקובץ לתוך מאגרי הזיכרון.
 - 2.1 שגיאה בעת העלאת קבצים/סריקת ה-SD תוביל לבדיקה שתעשה על מחזור זמן מסוים עד שהמערכת תצליח להוציא את המבוקש, ובכך תחזור חזרה אל בלוק 2.0
3. בלוק ניהול ממשק המשתמש, האחראי לעדכן את כל הקשור עם האינטראקציה של המשתמש למערכת עצמה, הן פלט חזותי והן קלט פיזי.
 - 3.1 בלוק בקרה וניהול פרמטרים: מאחסן את המצב הנוכחי של הפרמטרי הניתנים להתאמה על ידי המשתמש, ואת המצב העכשווי של האקולייזר. מנהל גישה בטוחה לפרמטרים משותפים בין

בלוק בקרת הממשק למשתמש לבלוק צינור עיבוד השמע. בעזרת מניעה הדדית לתנאי מירון כאשר משימת ממשק המשתמש מעדכנת ערך הגברה בזמן שמשימת השמע קוראת אותו לפעולת הפילטרים.

3.2. בלוק בקרה של ממשק משתמש, מנהל התצוגה שבעת כל קריאה ועדכון פרמטרים, הצג מורה על המצב העכשווי של הפרמטרים והאקולייזר-אינדיקציה חזותית למצב שבו עומדת המערכת והערכים המוכלים בה.

4. בלוק צינור עיבוד שמע, אחראי על יישום העבודה החישובית של השבב-פונקציות DSP, השמת הפילטרים-על החציה (buffer) המכיל את נתוני השמע.

4.1. בלוק פלט השמע דרך מגבר ורמקול: לאחר הפעלת האקולייזר על השמע, המידע מועבר על המגבר בממשק התקשורת המתאים ומשם מתמיר את המידע להפעלתו וניגונו ברמקול עצמו.

• אסטרטגיית תכנון התוכנה

אסטרטגיית עיצוב התוכנה הכוללת מתמקדת בהבטחת עיבוד שמע אמין בזמן אמת תוך שמירה על ממשק משתמש רספונסיבי. החלטות אסטרטגיות מרכזיות כוללות את בחירת סביבת הפיתוח ושקול היטב אילוצי זמן אמת.

- בחירת סביבת פיתוח (ESP-IDF)

בחינת סביבת פיתוח, הבחירה ב-ESP-IDF, Espressif IoT Development Framework, כל פני Arduino IDE כרוכה בכמה סיבות. בעוד שסביבת הפיתוח של ארדואינו מספק שכבת הפשטה פשוטה יותר שמזרזת את הפיתוח הראשוני עבור פרויקטים רבים, ESP-IDF מציע שליטה וגמישות ישירה יותר, דבר שמועיל מאוד עבור יישומים עתירי חישובים וצורך בתגובות גבוהה בזמן אמת. יתרונות השימוש ב-ESP-IDF עבור הפרויקט הינם בעיקר סביב ביצועי המערכת הכוללים:

- גישה ישירה ל-FreeRTOS, ומספק גישה מלאה לתכונות שלו שלו כמו יצירת משימות (אשר ניתן לממש במלואו את המעבד בעל שתי הליבות), תזמון, תקשורת בין משימות וסנכרון. בקרה זו חיונית לניהול פעולות בו-זמניות כמו השמעת שמע וטיפול בממשק משתמש באופן יציב ועמיד.
- מנהלי התקנים היקפיים אופטימליים: ESP-IDF מספק מנהלי התקנים ב-low level, המציעים אפשרויות תצורה רבות יותר וביצועים פוטנציאליים (אם משתמשים בדרישות אליהם, סוג משתנים, אורכי מבני נתונים ועוד) בהשוואה לספריות Arduino שהן כלליות יותר. זה מאפשר כיוון מרוכז יותר כמו I2S לתפוקת שמע בצורה אופטימלית.
- ביצועים ואופטימיזציה: מאפשר לעבוד קרוב יותר לחומרה ומספק כלים (כמו menuconfig, רישום לוגים מפורט ESP_LOGI) לאופטימיזציה של ביצועים ושימוש בזיכרון, ובין היתר יישום אלגוריתמי DSP עתירי חישוב.
- גישה לספריות ספציפיות: זה מאפשר שילוב Espressif ייעודיות, מתאימות לניצול חומרת הבקר בצורה המירבית, לשימוש בפרויקט שלנו הוא ספריית esp-dsp המשמשת ליישום יעיל של פילטר biquad. מסיבות אלה, וניסויים והשוואות שיתוארו בהמשך, נבחרה סביבת הפיתוח הנ"ל על מנת לעמוד בביצועים הנדרשים במשאבים המוגבלים הנתונים לנו.

- שיקולי עיבוד בזמן אמת

הדבר מתרכז בהבטחת השמעת שמע חלקה וללא הפרעות תוך כדי יישום השפעות האקולייזר-דבר הדורש תשומת לב לאילוצים בזמן אמת. האתגר המרכזי הוא לבצע ברציפות את הזרימה הבאה: קריאת נתוני שמע <- החלת הפילטרים <- נתוני פלט, מהר ממה שנתוני השמע מצרכים על ידי התקן הפלט-כדוגמה בקצב דגימה של 44.1 קילוהרץ, חציה (buffer) של 512 דגימות, משמע כל רצף זה צריך להתבצע מתחת ל:

$0.00072 \text{ seconds} \approx 32 * 44100^{-1}$. אי עמידה בסף זה תגרום לחוסרים במאגר יציאת ה-I2S שגורם לשמע סטטי או בעיות שמע. שיקולים מרכזיים המטופלים באסטרטגיית התכנון כוללים:

- עומס חישובי: החלת חמישה פילטרים biquad מסדר שני על כל דגימת שמע מייצג עומס חישובי משמעותי. מעבד שנתון לנו אמנם עוצמתי ועומד בפני עצמו תיאורטית בדרישות, אך אלגוריתם הסינון חייב להיות יעיל מספיק כדי לעבד כל חציצה היטב בתוך הזמן הדרוש. שימוש בפונקציות אופטימיזציה שמציעות מירבית עבור אותם שבבים משובץ מחשב, מסייע להשיג את המטרה הזאת.
- ניהול מקביליות: שימוש נכון בליבות המעבד, טיפול בקלט המשתמש(קריאות אנלוגיות ועדכון הפרמטרי ותצוגה) במקביל לצינור השמע הינה קריטריון שנרצה לענות עליו. כדי להבטיח שמימות כאלה או אחרות לא יעלו בעדיפות בפרויקט זה, נגדיר את סדר העדיפויות של המשימות כך שנבטיח כי צינור השמע בעל העדיפות הגבוהה לא מתעכב יתר על המידה על ידי משימות בעלות עדיפות נמוכה כמו עדכוני ממשק המשתמש.
- הגנה על משאבים משותפים: פרמטרים כמו הגבר של פס האקולייזר נגישים הן על ידי משימת עיבוד השמע(כדי להחיל את מקדמי הסינון הנכונים) והן על ידי משימת ממשק המשתמש(כדי לעדכן על סמך קלט הפוטנציומטרים). גישה לא מותאמת עלולה להוביל להתנהגות בפילטר שאינה עקבית, לכן ניצור מניעה הדדית-mutex וכך נבטיח כי רק משימה אחת תוכל לגשת או לשנות נתונים משותפים אלה, תוך מניעה תנאי מירוץ ושמירה על שלמות הנתונים.

ישום תוכנה

• ארכיטקטורת תוכנה

סעיף זה מתעמק ביישום הקונקרטי של ארכיטקטורת התוכנה, ומתאר את זרימת הבקרה העיקרית, המשימות וזרימת הנתונים בתוך תוכנה הפועלת בבקר ESP32-S3 איתו אנו משתמשים

- לולאת בקרה ראשית

זרימת הביצוע העיקרית מתחילה בפונקציה `app_main`, נקודת הכניסה הסטנדרטית עבור יישומי ESP-IDF. מבצעת תחילה אתחולים חיוניים: הרכבת מערכת הקבצים של כרטיס ה-SD דרך `mount_sdcard`, הגדרת ציוד היקפי I2S לפלט שמע באמצעות `i2s_init`, הגדרת ה-ADC לקריאת פוטנציומטרים באמצעות `init_adc`. לאחר מכן מזהה את קבצי ה-`wav` הזמינים בכרטיס ה-SD באמצעות `find_wav_files`, והשלב המפצל את התהליך המרכזי למנהלי הממשקים שעליהם דיברנו לפני כן: ראשית ליצור את המנועול למניעה הדדית ותהליך יצירת משימת `freertos` נפרדת, שמה היא `control_update_task` באמצעות `xTaskCreate` כדי לטפל בממשק המשתמש ובצינור פלט שמע בו זמנית. לאחר יצירת המשימה, `app_main` נכנס ללולאת ההפעלה הראשית: הוא עובר על רשימת קבצי ה-`wav` שנמצאו, עבור כל קובץ קורא לפונקציה `audio_play_wav`, אשר חוסמת עד שהקובץ סיים להתנגן. ההפעלה הרציפה הזו של כל הקבצים שנמצאו מהווה את התנהגות הלולאה הראשית במימוש זה.

```
void app_main(void)
{
    mount_sdcard();
    i2s_init();
    init_adc();

    static char wav_files[MAX_WAV_FILES][MAX_FILENAME_LEN];
    int wav_count = find_wav_files("/sdcard", wav_files, MAX_WAV_FILES);
    ESP_LOGI(TAG, "Found %zu WAV files", wav_count);
    for (int i = 0; i < wav_count; i++){
        char filepath[128];
        snprintf(filepath, sizeof(filepath), "/sdcard/%s", wav_files[i]);
        ESP_LOGI(TAG, "wav %d: %s", i, filepath);
    }

    s_band_gain_mutex = xSemaphoreCreateMutex();
    xTaskCreate(parameters_update_task, "control_update_task", 4096, NULL, 5, NULL);

    for (int i = 0; i < wav_count; i++) {
        char filepath[128];
        snprintf(filepath, sizeof(filepath), "/sdcard/%s", wav_files[i]);

        ESP_LOGI(TAG, "Now playing: %s", filepath);
        audio_play_wav(filepath);
    }
}
```

- תיאורי מודלים

- התוכנה פועלת באמצעות שני הקשרים ראשיים של ביצוע המנוהלים על ידי FreeRTOS:
משימה ראשית-שמע: הקשר זה פועל באופן עקיף בלולאת הפונקציה `app_main` לאחר האתחול. תחומי האחריות שלו כוללים איטרציה של קבצי `wav` וביצוע הפונקציה `audio_play_wav`. בתוך הפונקציה הזאת, ישנו טיפול בקריאת כרטיס הזיכרון, מפעיל את עיבוד ה-DSP על ידי קריאה לפונקציית עזר שנרחיב עליה בהמשך, ושולח נתוני שמע המעובדים למכשיר ההיקפי I2S. משימה זאת אחראית על כל צינור השמע מקלט הקובץ ועד פלט החומרה.
- **משימת שליטת עדכון ממשק המשתמש:** משימה זו נוצרת ופועלת במקביל למשימת השמע הראשית, אשר מוגדרת להיות צמודה לליבה במס' סידורי 1 בעוד שהמשימה הראשית תמצא במס' סידורי 0 (הוגדר בהגדרות המוצעות על ידי Espressif ב-menuconfig). הוא קורא מדי מחזור זמן מסוים את ערכי ה-ADC משלושת הפוטנציומטרים, ממפה קריאות אלו לאינדקס לעוצמת שמע, אינדקס פס התדרים וההגבר ומעדכן בבטחה את מצב האקולייזר, ומטפל בעדכוני תצוגה בצורה חזותית למשתמש. הוא כולל לוגיקה על ידי חתימות זמן כדי לקבוע איזה פרמטר ברצון המשתמש להתאים, תוך הבטחה שההגבר מוחל על הפס תדר המיועד.

- זרימת נתונים

- תנועת זרימת הנתונים דרך המערכת עוברת בנתיבים נפרדים שאחד נחתך לשני:
זרימת נתוני השמע: נתוני שמע PCM גולניים של 16 סיביות מגיעים מקובץ `wav` בכרטיס ה-SD. הם נקראים בלוק אחר בלוק לתוך מאגר ביניים (חציצה `audio_buffer` בתוך פונקציית `audio_play_wav`). מאגר זה עובר עיבוד השמע והמרת סוג המשתנה לשימוש בפונקציות DSP בצורת אופטימלית, החלת האקולייזר-עובר 5 פילטרים מותאמים לפי עוצמת הקול, ומומרים חזקה לתוך אותו מאגר. חציצה מעובדת זאת מעוברת לאחר מכן לפרוטוקול הנתונים של I2S אל מגבר max98357a ומשם אל הרמקול.
- **זרימת נתוני פרמטרים ונתוני התצוגה:** מתחים אנלוגים מהפוטנציומטרים נקראים בעזרת הספריה `adc_oneshot_read`. ערכים דיגיטליים גולמיים אלו ממופים לפרמטרים ברמת היישום:
`freq_idx`-אינדקס פס התדר, `gain_idx`-אינדקס ההגבר, `s_volume`-עוצמת הקול. לאחר שנבחרו הפרמטרים היא מעדכנת את האלמנט המתאים במערך המעמד של האקולייזר כדי להחיל את קנה המידה הנכון של עוצמת הקול ולבחור את מקדמי הסינון המחושבים מראש המתאימים עבור כל פס(יתואר בהמשך את דרך החישוב והיישום). ערכי הפרמטרים הנוכחים והמעמד הנוכחי של האקולייזר נקראים ומיוצרים לפלט מעוצב עבור הצג, הנשלחים באמצעות פקודות/נתונים בתקשורת I2C לבקר התצוגה.

• אלגוריתם ליבה-החלת הפילטרים וניהול השמע

סעיף זה תתרכז ביישום ורשימת התוכנה עצמה, איך בוצעה ואיך מומשה בתור הבלוקים שהוסברו ותוארו, פונקציונליות המודלים וזרימת הנתונים בתהליך הריצה עצמו.

- חישוב מקדמי הפילטרים

כפי שהוסבר, עבודה במיקרו-בקר ESP32-S3 כרוך בהגבלות ביצוע שתצטרך לעמוד בהן, במקום לבצע ביצוע חישוב נוסף על מאפייני הפילטרים, חמישה חישובים בכל מחזור זמן שבו תהליך השמת הפילטרים על חציצת השמע מתבצע. ייצור אקולייזר גרפי יהיה על ידי הגברים קבועים על חמישה פסי התדר-משמע החישובים של המקדמים יעשו מראש ויעלו אל זכרון השבב. חישוב של כל אחד מסוגי הפילטרים, תלויי הפרמטרים שבהם

אנחנו משתמשים במערכת, ובמשתנים הבאים: הגבר (G), ערך אמצע פס התדר (f_c), קצב הדגימה (f_s) וקבוע האיכות (Q), על ידי מימוש הנוסחאות הבאות שחושבו ב-python:

- חישוב מקדמי פילטר מדף (shelf):

```
...
Designing High/Low Shelf Filter
Calculating the filter's coefficients and visualize its frequency response
...
def shelving_coefficients(fc, fs, G, mode): # without Q
    pi = np.pi
    k = np.tan((pi*fc)/fs)
    v0 = 10**(G/20)
    r2 = sqrt(2)
    if G < 0: # cut

        if mode == 'low': # low cut
            d = v0 + r2*sqrt(v0)*k + k**2
            a1 = 2 * (k**2 - v0) / d
            a2 = (d - 2 * r2*sqrt(v0)*k) / d
            b0 = (v0 * (1 + r2*k + k**2)) / d
            b1 = (2 * v0 * (k**2 - 1)) / d
            b2 = (v0 * (1 - r2*k + k**2)) / d

        else: # high cut
            d = 1 + r2*sqrt(v0)*k + v0 * (k**2)
            a1 = 2 * (v0 * k**2 - 1) / d
            a2 = (d - 2 * r2*sqrt(v0)*k) / d
            b0 = (v0 * (1 + r2*k + k**2)) / d
            b1 = (2 * v0 * (k**2 - 1)) / d
            b2 = (v0 * (1 - r2*k + k**2)) / d

    elif G > 0: # boost
        if mode == 'low': # low boost
            d = 1 + r2*k + k**2
            a1 = 2 * (k**2 - 1) / d
            a2 = (d - 2 * r2 * k) / d
            b0 = (1 + r2*sqrt(v0)*k + v0 * (k**2)) / d
            b1 = 2 * (v0 * k**2 - 1) / d
            b2 = (1 - r2*sqrt(v0)*k + v0 * (k**2)) / d

        else: # high boost
            d = 1 + r2*k + k**2
            a1 = 2 * (k**2 - 1) / d
            a2 = (d - 2*r2*k) / d
            b0 = (v0 + r2*sqrt(v0)*k + k**2) / d
            b1 = 2 * (k**2 - v0) / d
            b2 = (v0 - r2*sqrt(v0)*k + k**2) / d

    return [b0, b1, b2, 1, a1, a2]
```

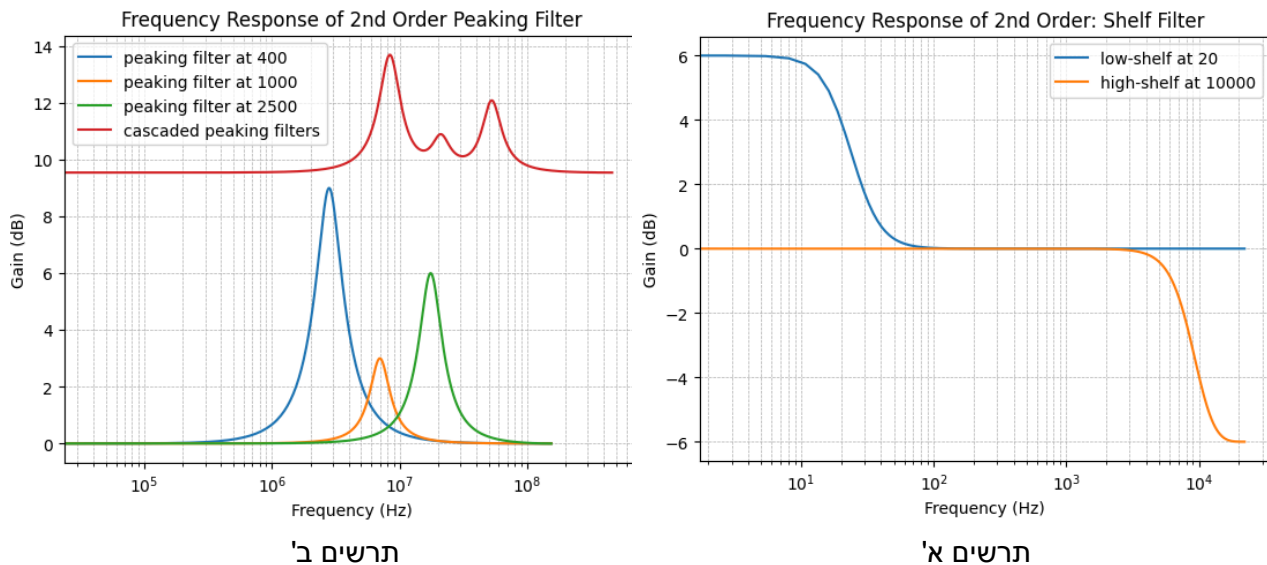
- חישוב מקדמי פילטר שיא (peaking):

```
Designing Peaking/Notch Filter
Calculating the filter's coefficients and visualize its frequency response
...
def peaking_coefficients(fc, G, Q, fs):
    pi = np.pi
    k = np.tan((pi*fc)/fs)
    v0 = 10**(G/20)

    #if v0 < 1:
    #    v0 = 1/v0

    if G > 0: # boost
        d = 1 + k/Q + k**2
        b0 = (1 + ((k*v0)/Q) + k**2) / d
        b1 = (2 * ((k**2) - 1)) / d
        b2 = (1 - ((k*v0)/Q) + k**2) / d
        a1 = b1
        a2 = (1 - (k/Q) + k**2) / d
    else: # cut
        d = 1 + (k/(Q*v0)) + k**2
        b0 = (1 + (k/Q) + k**2) / d
        b1 = (2 * ((k**2) - 1)) / d
        b2 = (1 - (k/Q) + k**2) / d
        a1 = b1
        a2 = (1 - (k/(Q*v0)) + k**2) / d
```

על מנת לבדוק האם התוצאה הרצויה אכן התקבלה, נציב את הפילטר בפונקציה המציעה הספריה `scipy` ונחשב את תגובת התדר (frequency response) של כל אחד מהפילטרים (ראה תרשים א'), ונבדוק את ההשפעה של 3 פילטרי שיא על פסי תדר המיועדים שלקחנו לפרויקט זה, כאשר לכל אחד יש הגבר שונה (תרשים ב', ניתן לראות את תגובת התדר של כל פילטר בנפרד; בצד העליון של הגרף יהיה ניתן לראות כי הסכמה ביניהם תהיה תוצאה רצויה, הגברה משולבת של אותם פסי תדר בטווח הנ"ל). הפילטרים יושמו על שירים בפורמט קובץ `wav`, ועל פי ייצוג ספקטוגרמה היה ניתן לראות כי אכן בתדרים המוגברים הייתה עלייה באמפליטודה כרצוי. כמו כן המעטת הגבר בתדרים מסוימים.



- יישום פילטר בזמן אמת (ליבת עיבוד DSP)

בהינתן מצב מסוים בו האקולייזר נמצא, ניישם את הפילטרים של כל פס תדר על ידי פונקציית עיבוד אותו דיגיטליים-DSP, אשר בה נתמקדנה. ראשית, פונקציות המוצעות על ידי הספריה `esp_dsp` הינן פונקציות שעובדות בעילות מירבית, מתרכזות אופטימיזציה חישובית במשאבים הנתונים, כאשר נמסרים לה סוג נתונים מתאימים-משתנים מסוג `float32`. לאחר וקריאה מקובץ מקור ממלאת חציצה של `int16_t` (מספר שלם בעל עומק סיבית של 16), ונצטרך להמיר את הנתונים בהתאם לשימוש נכון בקריאה לפונקציית ה-DSP. לאחר ומעמד האקולייזר הינו משתנה משותף בין שני תהליכים, נמנע הדדית שימוש/שינוי בו"ז שהיה יכול לגרום לאי עקביות בתדר-דבר שנרצה למנוע. הקוד להלן:

- פעולה: מיישם את הפילטרים מתאימים על פי מעמד האקולייזר הנמצא במערכת.
- קלט: מצביע אל חציצה בגודל 512 משתנים מסוג מספר בעל 16 סיביות, מספר הדגימות בפועל שהועברו.
- פלט: הפונקציה אינה מחזירה דבר, משנה את תוכן החציצה מנתוני השמע המקוריים שנקראו אל תוכן השמע מושפע על ידי התנהגות האקולייזר שהיה בתחילת הרצת הפונקציה.

```
void audio_filter_process(int16_t *samples, size_t sample_count)
{
    static float float_buffer[512];
    for (size_t i = 0; i < sample_count; i++) {
        float_buffer[i] = (float)samples[i] / 32768.0f;
    }
}
```

```

}
float coeffs[5];
int band_gains_local[5];

if (xSemaphoreTake(s_band_gain_mutex, portMAX_DELAY) == pdTRUE) {
    memcpy(band_gains_local, s_band_gains_db, sizeof(band_gains_local));
    xSemaphoreGive(s_band_gain_mutex);
}

for (int i = 0; i < 5; i++){
    memcpy(coeffs, biquad_coeffs[i][band_gains_local[i]], sizeof(coeffs));
    dsps_biquad_f32(float_buffer, float_buffer, sample_count, coeffs,
                    filter_state[i]);
}

for (size_t i = 0; i < sample_count; i++) {
    float scaled = float_buffer[i] * s_volume * 32767.0f;
    if (scaled > 32767.0f) scaled = 32767.0f;
    else if (scaled < -32768.0f) scaled = -32768.0f;
    samples[i] = (int16_t)scaled;
}
}

```

לאחר המרה נוספת בחזרה לסוג המשתנה int16_t יהיה ניתן להחזיר את השמע המושפע אל הכתובת החציה שהועברה.

- זרימת נתוני שמע

- כפי שהוזכר-לולאת הבקרה הראשית, לאחר אחזור(fetch) קובץ מכרטיס זיכרון, קוראת לפונקציה `audio_play_wav` המקבלת path אבסולוטי אל הקובץ שמכיל את נתוני השמע ומתחילה לקיים את לוגיקת זרימת נתוני השמע דרך מעגל השלבים הבאים:
1. קריאה אל תוך חציה בגודל 512 מסוג int16_t מהקובץ wav הנוכחי שאותו אנחנו מגנים.
 2. העברת מצביע על חציה זאת אל הפונקציה להחלת הפילטרים, שלאחריה כמתואר, מתקבל נתוני שמע שמוחלים עליהם השינויים(פילטרים והשמת עוצמת השמע).
 3. כתיבה לערוץ התקשורת בממשק I2S עם המגבר max98357a ומשם יוצא הניגון אל הרמקול.
- לאחר סיום ניגון קובץ, הוא נסגר והתהליך חוזר אל לולאת הבקרה הראשית.
- פעולה: אחראי על ניגון קובץ wav מתחילתו ועד סופו, תוך החלת פילטרים והשפעת עוצמת הקול.
 - קלט: כתובת אבסולוטית של קובץ wav שנרצה לנגן.
 - פלט: אין.

להלן פונקציית `audio_play_wav`:

```

void audio_play_wav(const char *filepath)
{
    FILE *fp = fopen(filepath, "r");
    if (!fp) {
        ESP_LOGE("WAV_PLAYER", "Failed to open file");
        return;
    }
    wav_header_t header;
    fread(&header, sizeof(header), 1, fp);
}

```

```

int16_t audio_buffer[512];
size_t bytes_read, bytes_written;
while ((bytes_read = fread(audio_buffer, 1, sizeof(audio_buffer), fp)) > 0) {
    audio_filter_process(audio_buffer, bytes_read / 2);
    i2s_channel_write(tx_channel, audio_buffer, bytes_read, &bytes_written,
                      portMAX_DELAY);
}
fclose(fp);
}

```

• לוגיקת עדכון ממשק משתמש

במקביל לתהליך זרימת נתוני השמע, בליבה השנייה שיש בשבב עליו אנחנו עובדים-רץ תהליך המיישם את לוגיקת תחזוקת ממשק המשתמש בהחלת השינויים שנעשו על אופן התנהגות האקולייזר והמערכת. ממשק המשתמש מתחלק לשני חלקים, האחד הוא ניהול הקלט של המשתמש על ידי קריאות הפוטנציומטרים, השמת הנתונים באופן בטוח שהשינויים לא יגרמו נזק לתפקוד המערכת בזמן אמת. השני הוא לנהל את התצוגה אל המשתמש: מצב הנוכחי של האקולייזר וערכי הפרמטרים הנקראים מהחומרה.

- קריאות רכיבי החומרה ומיפוי פרמטרים

קריאות הפוטנציומטרים נערכות עבור כל מחזור זמן מוקצב על ידי המוגדר בפונקציית `parameters_update_task`, עבור שלושת הפוטנציומטרים המיועדים לשלושת הפרמטרים שעליהם אחראים-עוצמת שמע, פס התדר הנוכחי, הגבר פס התדר. כל קריאה מפוטנציומטר יכול להפיק קריאה של ערכים החל מ-0 עד ל-4095 בסוג משתנה FLOAT, עבור כל אחד מהערכים מקבל מיפוי מתאים על ידי הפונקציה MAPADC שעל פי רמות שאנחנו מעבירים בארגומנטים של הפונקציה הנ"ל. נצטרך למנוע שני תקלות שנוכל להיתקל בהן ביישום לא נכון בלוגיקת הקריאה:

1. דריסת נתונים במצב האקולייזר-כאשר משנים בחופשיות את ערך פס התדר ומחילים את השינוי בכל איטרציה, נקבל כי ההגבר שנקרא בפוטנציומטר מיושם על כל פס תדר שנקרא בפוטנציומטר-משמע עבור כל פס תדר נצטרך להגדיר מחדש הגבל של כל פס. הפתרון של תקלה זאת היא ליישם שתי חתימות זמן על כל שינוי פוטנציומטרים, וכך נדע את כוונת המשתמש האם ירצה לשנות את הפס תדר או את ההגבר לאותו פס תדר, בהתאם לתנאים שצריכים להתקיים עבור כל אחד מההתרחשויות.
2. שימוש ושינוי תוכן מצב משותף האקולייזר-על מנת למנוע מצב כזה ששינוי בפונקציה זו תתרחש באותה הזמן עם החלת החלת האקולייזר על השמע, ניצור מנעול מניעה הדדית, אשר כל פונקציה בדרכה שומרת על החזקה קצרה על מנעול מניעה הדדית זאת שלא תמנע מאחת התהליכים. לכן בפונקציה זאת, כאשר שינוי זה חל הפונקציה מונעת מהתרחשויות ושימושים נוספים במשתנים שבהם משתמשת על מנת לשנות את מצב האקולייזר. עבור כל איטרציה הפונקציה תעדכן את הממשק החזותי של המשתמש ע"י קריאה ל-

- פעולה: מנהל את בסיס ממשק המשתמש עבור קריאות קלט המשתמש מהחומרה ואוריינטציית הפעולות הנעשות בלולאת הבקרה המשנית.

- קלט: אין.

- פלט: אין.

להלן פונקציית `parameters_update_task`:

```

void parameters_update_task(void* arg)
{
    int prev_gain_idx = 3;

```



```

int prev_freq_idx = 0;
int64_t gain_ts = esp_timer_get_time();
int64_t freq_ts = gain_ts;

while (1){
    int adc_raw_volume, adc_raw_gain, adc_raw_freq;
    if (adc_oneshot_read(s_adc_handle, POT_ADC_CHANNEL_VOLUME, &adc_raw_volume) ==
        ESP_OK)
        s_volume = (float)adc_raw_volume / 4095.0f;
    // Read gain
    if (adc_oneshot_read(s_adc_handle, POT_ADC_CHANNEL_GAIN, &adc_raw_gain) ==
        ESP_OK)
        gain_idx = map_adc_to_level(adc_raw_gain, 7);
    // Read frequency
    if (adc_oneshot_read(s_adc_handle, POT_ADC_CHANNEL_FREQ, &adc_raw_freq) ==
        ESP_OK)
        freq_idx = map_adc_to_level(adc_raw_freq, 5);

    if (gain_idx != prev_gain_idx){
        gain_ts = esp_timer_get_time();
        prev_gain_idx = gain_idx;
    }
    if (freq_idx != prev_freq_idx){
        freq_ts = esp_timer_get_time();
        prev_freq_idx = freq_idx;
    }

    if (freq_ts < gain_ts){
        if (xSemaphoreTake(s_band_gain_mutex, portMAX_DELAY) == pdTRUE){
            s_band_gains_db[freq_idx] = gain_idx;
            xSemaphoreGive(s_band_gain_mutex);
        }
    }

    vTaskDelay(pdMS_TO_TICKS(250));
    print_ascii_eq();
}
}

```

- עדכוני צג

בכל איטרציה התכנית מעדכנת את פלט הממשק משתמש שנמצא על הצג-ברגע זה הפלט הוא חזותית בצורה גרפית שמיוצאת בפלט הסריאלי של התכנית. מציגה בגרף היסטוגרמי את מצב האקולייזר בחמשת פסי התדר הנתונים, אשר בכל אחד מהערכים יש אינדיקציה על איזה הגבר מיושם באותו פס תדר. בנוסף על כך, ערך עוצמת הקול, הפס תדר וההגבר בנבחרו גם הם מוצגים. הלוגיקה של הפלט בצורה דומה לה של הצגה על הצג-קוראת את ההגברים הנוכחיים מ-`s_band_gains_db`. היא ממפה את מדדי ההגבר (0-6) לרמות תצוגה (1-7)

ולאחר מכן עוברת איטרציות על הרמות האנכיות פסים אופקיים. עבר כל תא ברשת הזאת, היא מדפיסה תו בהתאם-מדפיס תוויות עבור רמות dB וערכי עוצמה/תדר/הגבר נוכחיים מתחת לגרף. בתצוגה של הפרמטרים דבר דומה יתבצע, ממלא חציצה המכילה את הפלט הסריאלי ומעבירה את מאגר הנתונים אל הצג דרך אמצעי התקשורת I2C.

- פעולה: אחראי תצוגה גרפית עבור ממשק המשתמש.
- קלט: אין, יש צורך לגישה למצב האקולייזר והפרמטרים שנקראו.
- פלט: אין, שינוי התצוגה עבור המשתמש.

```
void display_eq_status(void)
{
    char line_buffer[40];
    int eq_levels[NUM_BANDS];
    int band_gains_local[NUM_BANDS];

    if (xSemaphoreTake(s_band_gain_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        memcpy(band_gains_local, s_band_gains_db, sizeof(band_gains_local));
        xSemaphoreGive(s_band_gain_mutex);
    } else {
        ssd1306_display_text(&s_display_dev, 0, "EQ Mutex Busy!", 14, false);
        ssd1306_show_buffer(&s_display_dev);
        return;
    }

    for (int i = 0; i < NUM_BANDS; i++)
        eq_levels[i] = band_gains_local[i] + 1;

    for (int level = MAX_HEIGHT; level > 0; level--) {
        char* p = line_buffer;
        int remaining_len = sizeof(line_buffer);
        int written;

        if (level == MAX_HEIGHT)
            written = snprintf(p, remaining_len, " 9dB - ");
        else if (level == (MAX_HEIGHT / 2) + 1)
            written = snprintf(p, remaining_len, " 0dB==");
        else if (level == 1)
            written = snprintf(p, remaining_len, "-9dB - ");
        else
            written = snprintf(p, remaining_len, "      - ");
        p += written;
        remaining_len -= written;

        for (int band = 0; band < NUM_BANDS; band++) {
            if (eq_levels[band] == level)
                written = snprintf(p, remaining_len, "=");
            else
                written = snprintf(p, remaining_len, " ");
            if (written >= remaining_len)
                break;
            p += written;
        }
    }
}
```

```

        remaining_len -= written;
    }
    *p = '\0';

    ssd1306_display_text(&s_display_dev, MAX_HEIGHT - level, line_buffer,
strlen(line_buffer), false);
}

int current_gain_db = gain_levels_db[gain_idx];
const char* freq_str = freq_levels_hz_print[freq_idx];

snprintf(line_buffer, sizeof(line_buffer), "Vol:%2.0f% F:%sHz    ",
        s_volume * 100.0f,
        freq_str);
ssd1306_display_text(&s_display_dev, 7, line_buffer, strlen(line_buffer), false);

ssd1306_show_buffer(&s_display_dev);
}

```

• ניתוח מורכבות אלגוריתם

ישנם שני מכשולים עיקריים אשר אנחנו נתקלים ביישום התוכנה הזאת במיקרו-בקר הנתון לנו, שאחד הוא עומס חישובי של פילטרים בעזרת DSP, השני הוא אילוצי זמן אמת וניתוח הנתונים אשר עליהם נסביר ונרחיב את הבדיקות שנעשו על מנת להבטיח את יציבות המערכת הזמן ביצועה משימתה.

- עומס חישובי של מסנני DSP

עומס חישובי של הליבה נובא מיישום סוג הפילטרים biquad מדרגה מסוימת בפונקציה `audio_filter_process`. כל קריאה ל-`parameters_update_task` מבצעת כ-5 מכפלות ו-4 סכמות של משתנים מסוג float32 לכל דגימה מעובדת. מכיוון שנעשה עבור 5 רצועות, עלות הכוללת היא כ-25 מכפלות ו-20 סכמות לכל דגימת שמע-עבור חציצה בגודל של 512 דיגמות, זה מתורגם ל-12800 מכפלות ו-10240 סכמות לכל חציצה. כפי שניתן לראות, ישנה תקורה על המרת int <-> float, שינוי קנה המידה של עוצמת הקול. למרות שבביצוע איטרציה אחת של הפונקציה מעבד דו-ליבתי בקצב של 240MHz יכול להתמודד עם החישוב הנ"ל ומצויד היטב עם פונקציות הממוטבות המסופקות על ידי ספריית esp-dsp. הביצועים מספיקים תיאורטית לעיבוד בזמן אמת בקצב דגימה של 44.1kHz.

- אילוצי זמן אמת וניתוח תזמון

אילוצי זמנים העיקריים של המערכת הוא הצורך לספק נתוני שמע מעובדים באופן רציף למכשיר אליו מעבירים מידע בעזרת I2S ללא הפרעה(underrun). הזמן הזמין לעיבוד מאגר אחד בגודל של 512 דגימות הוא כ-11.6 מילישניות. בתוך מסגרת זמן זו, המערכת חייבת לקרוא את הבלוק הבא מכרטיס הזיכרון, לבצע סינון DSP, ולשלוח את המאגר לדרייברים הנתמכים על ידי I2S. שימוש בספריית FreeRTOS מאפשר למשימת השמע ולמשימת ממשק המשתמש(בעדיפות הנמוכה יותר) לפעול בו"ז. אופי החסימה של `i2s_channel_write` עם `portMAX_DELAY` מבטיח כי עיבוד השמע לא יפעל מהר יותר מקצב הפלו-הגורם הקריטי הוא להבטיח שכמות הזמן שמאחורי איטרציה אחת עבור מאגר יחיד בלוגיקת זרימת השמע יישאר בעקביות מתחת לזמן הנדרש, 11.6 מילישניות. בעיות תזמון אפשריות העלולות לבוע מגישה איטית לכרטיס הזיכרון או אם המעבד

עמוס מאוד על ידי משימות אחרות(בהינתן ורק משימת ממשק המשתמש קיימת כאן), דבר המשפיע על יכולה של משימת השמע לעמוד בסף הזמן.

תיעוד הערכות ובדיקות המערכת

סעיף זה מתאר את הליכי הבדיקה שבוצעו כדי להעריך את הביצועים והפונקציונליות של מערכת האקולייזר במימוש על המיקרו-בקר ESP32-S3, תוך התמקדות מיוחדת במשימה הראשית, עתירת החישוב של סינון דיגיטלי בזמן אמת. נפרטנה את תוכנית הבדיקה, התוצאות שהתקבלו, ניתוח תוצאות אלו, בסיס של סיבתיות הפיתוח והאתגרים שנתקלו בהם במהלך בפיתוח.

• תוכנית ונהלי בדיקה

נערוך סדרת בדיקות נהלים וקריטריונים שונים העוסקים במדדי ביצוע המערכת מבחינת זמנים, איכות השמע, חריגות אפשריות ונקודות שנצטרך לקחת לצומת הלב. ראשית, נראה את אופן הפעולה השונה שמציעים סביבות העבודה השונות-Arduino IDE לנגדו Espressif IoT development framework: ESP-IDF, נציג תיעודי ביצוע עבור קטעי קוד המבצעים את הפעולה המבוקשת בעל החישוב הגדול ביותר ואת הסקת המסקנות על פיהם.

לאחר שהתקבלה ההחלטה להמשיך עם מסגרת ESP-IDF וספריית esp-dsp עקב ביצועים מספקים, וכעת יוחל השלב השני של בדיקות. נראה את הפרשי הזמנים, וחריגות במקרה שהיו בתוכנית המלאה עבור פרמטרים הנוגעים לשמע-גודל מאגר, קצב הדגימה, זמני מחזור של לולאות הבקרה וכו'. כמו כן, בהמשך נציב בעיה המוטמנת בשיטת המניעה ההדדית ונראה כי היא זניחה במערכת הכוללת בה אנו משתמשים.

• תוצאות ניתוח

סעיף זה מתמקד בתיאור והסבר עבור ניסוי הערכות ובדיקות מערכת כאשר ניצבים בתנאים שונים, את הממצאים והמסקנות, התוצאות שהתקבלו מהליכי הבדיקה סיפקו הנחיות למשך הפיתוח וליישום המערכת הסופי.

- מדידת ביצועים

מבחני הביצועים ההשוואתיים הניבו ממצאים משמעותיים בנוגע לבחירות במסגרת והספריות. יישום ראשות בתוך Arduino IDE, באמצעות עקרונות פעולה של פילטר ממבנה bi-quad הן ידני והן בספריות מיובאות ל-Arduino, התקשו לעמוד בתפוסת המעבד בזמן ריצה המבוקש-זאת באופן עקבי בלוחות הזמנים הנדרשים לעיבוד חמישה שלבי סינון בקצב הדגימה הנדרש מבלי להעמיס כובד על ליבת מעבד ה-ESP32-S3, שהוביל לקראקים קוליים וחוסר יציבות בפלט השמע. לעומת זאת באמצעות מסגרת ESP-IDF ומינוף ספריית esp-dsp הראה ביצועים עדיפים באופן ניכר. התוצאות להלן(בכולם נעשו בגדול של 512 דגימות במאגר אחד):

ESP-IDF	Arduino IDE		
בעזרת esp-dsp	בעזרת esp-dsp חיצוני	יישום ידני	
ממוצע 1.9 מילישניות זמן גרוע ביותר 2.6 מילישניות	ממוצע 4.1 מילישניות זמן גרוע ביותר 7.3 מילישניות	ממוצע 22.6 מילישניות זמן גרוע ביותר 34.9 מילישניות	יישום פילטר אחד
ממוצע 9.6 מילישניות זמן גרוע ביותר 10.3 מילישניות	ממוצע 23.4 מילישניות זמן גרוע ביותר 42.1 מילישניות	-	יישום חמש פילטרים עוקבים

לפיכך, בחרנו לעסוק ולפתח את תוכנת הפרויקט בסביבת הפיתוח של esp-idf, בדיקות נוספות בתוך הסביבה בחנו את הפרשות של גודל המאגר; בעוד שמאגרים קטנים הגבירו את התקורה והקריאות לעיבוד, ומאגרים גדולים הגבירו את ההשהייה ותגובתיות על שינוי האקולייזר, הגודל הנבחר של 512 דגימות סיפק פשרה טובה, שאפשרה זמן מספיק לעיבוד תוך שמירה ל השהיה מקובלת.

- הערכת איכות שמע

מבחני האזנה סובייקטיביים היוו את הבסיס להערכת איכות השמע. לאורך הפיתוח ובמיוחד לאחר החלטת והתיישבות בסביבת פיתוח esp-idf פלט השמע נוטר לצורך בהירות, היעדר רעש ומזעור תקלות. אינדיקציה מרכזית לבעיות כמו קליקים, פקיעות, עיוות ורעש סטטי הואזנו בקפידה, במיוחד במהלך מעברים או תקופות של עומס עיבוד גבוה. נמצא כי המימוש הסופי שרץ על ESP-IDF מספק פלט שמע מונופוני ברור, רציף ללא תקלות במהלך השמעת קבצי ה-wav עם החלת האקולייזר, מה שנתן את היציבות והורה על יכולת המערכת לעמוד בדרישות העיבוד בזמן אמת מבלי לפגוע בחווית האזנה.

- אימות התאמות פס באקולייזר

בדיקות האימות התמקדו בהבטחת שפונקציונליות האיקוויזציה הליבה פועלת כמתוכנן-ספציפית שהתאמת הגבר עבור פס תדרים נבחר יצרה את השינוי הצפוי בפלט השמע. באמצעות רצועות שמע שונות עם תוכן תדרים שונה(אשר כל קבוצת תדרים מתרכזות בג'נרות שונות-ג'ז ומטאל יתרכזו יותר Bass~Mid range בעוד פופ ומוזיקה אלקטרונית תתמקד ב-Treble) ולאחר מכן התאמת פוטנציומטר ההגבר בטווח שלו. מבחני ההאזנה נעשו על ידי בדיקת תגובת התדר של אותם שירים מול השיר שתנגן המערכת באותם תנאים. זה אימת שפימוי הפאמרטים מהפוטנציומרטים, בחירת מקדמי הסינון המתאימים המבוססים על קלט המשתמש, ויישום הפילטרים הללו בליבת ה-DSP פעלו כהלכה וכדי לשנות את ספקטרום השמע כפי שתוכנן על ידי האקולייזר.

• אתגרים שנתקלנו בהם ופתרונות

האתגר העיקרי במהלך הפיתוח היה השגת ביצועים עמידים ויציבים בזמן אמת עבור יישום חמישה פילטרים בו זמנית במיקרו-בקר בו אנו משתמשים. ניסיונות ראשוניים באמצעות מסגרת Arduino IDE, במיוחד עם אלגוריתם סינון המיושם באופן ידני, התבררו כיקרים מדי מבחינה חישובית, וכתוצאה מכך נגרמו תקלו שמע או דרישת משאבי CPU מעל הסף המותר. צוואר הבקבוק זה היה חייב חקירה וניסוי של גישות חלופיות. הפתרון כלל את הבדיקות השוואתיות שתוארו קודם לכן שהדימו בבירור את יתרונות הביצועים של העברת הפרויקט למסגרת הפיתוח של ESP-IDF ושימוש בספריית esp-idf המותאמת המסופקת על ידי Espressif. אתגר נוסף היה ליישם מערכת עם רבת משימות הניגשות למשתנים משותפים, היה הבטחת שלמות הנתונים עבר פרמטרי האקולייזר תוך שמירה על יציבות השמע. הפונקציה `parameters_update_task` משנה את הגדרות ההגבר בהתבסס על קלט המשתמש, הבעוד שמשימת השמע הראשית קוראת הגדרות אלה כדי להחיל את המסננים הנכונים. ללא סנכרון נכון, משימת השמע עלולה לקרוא ערכים לא עקביים או מעודכנים חלקית, מה שמוביל לסינון שגוי. מצב זה טופל על ידי יישום Mutex אשר הבטיח גישה למערך המשתנות תהיה אטומית, ובכך מונע תנאי מירוץ בין משימת הממשק המשתמש לבין צינור עיבוד השמע. נוסף על כך, יישום של mutex היה יכול לגרום לאיחור ועיכוב מילוי מחסנית השמע המוחזקת לתקשורת מול המגבר בעל התקשורת SPI. מפתר על הקטנת הפעולות ומזעור העבודה הנעשית בתפיסת ה-mutex.

• הישגי הפרויקט

פרויקט זה הגשים בהצלחה את מטרתו המרכזית-תכנון, יישום ובדיקה של אקולייזר גרפי ונגן שמע פונקציונלית המתמקדת במיקרו-בקר ESP32-S3. בין ההישגים המרכזיים נמנה פיתוח נגן שמע המסוגל לקרוא ולעבד קבצי wav מונופוניים בעלי עומק של 16 סיביות שנדגמו בקצב של 44.1 קילוהרץ בכרטיס זיכרון microSD. הישג מרכזי היה יישום בזמן אמת של האקולייזר בעל 5 פסים באמצעות מסנני IIR biquad מדירוג שני, תוך מינוף ספריית esp-dsp המותאמת במסגרת ESP-IDF. בדיקות ביצועים הראו באופן בעל בסיס שגישה זו יעילה משמעותית מיישומים אחרים, אשר מאפשר פעולה יציבה בזמן אמת. הפרויקט כולל ממשק משתמש פונקציונלי המשתמש בשלושה פוטנציומטרים לשליטה על עוצמת הקול, בחירת תחומי תדרים והתאמת הגברה בתוך אותך פסים. אתגרים במקביליות בין משימות שונות בתוכנת הפרויקט-צינור השמע וממשק המשתמש טופלו ביעילות באמצעות שימוש בספריות המאפשרות כך וסנכרון מניעה הדדית, מה שהבטיח שלמות נתונים. הערכות איכות שמע ובדיקות פונקציונליות הראו כי המערכת מספקת פלט שמע ברור במזעור תקלות אפשרויות ואכן השפעת האקולייזר משנים כשורה את ספקטרום התדרים בהתאם לקלט המשתמש.

• עבודה עתידית ושיפורים פוטנציאליים

בעוד שהמערכת נוכחית מדגימה פונקציונליות ליבה מוצלחת, מספר דרכים לפיתוח עתיד לשיפור את יכולת המערכת ואת חווית המשתמש.

- פלט סטריאו

שיפור משמעותי יהיה הוספת תמיכה בשמע סטריאופוני. הדבר ידרוש שינוי בתוכנה לפיתוח נכון של נתוני ה-wav, מה שעלול להכפיל את עומס העבודה של ה-DSP אם מיושם שוויון ערוצים עצמאי(או החלת אותה עקומה לשני הערוצים). התצורה ההיקפית של I2S תצטרך התאמה, ויהיה צורך להרחיב את חומרת הפלט כך שתכלול מגבר סטריאו או רכיב max98357a ורמקול. הרחבה זו תגרור הערכה מחודשת של קיבולת הביצועים בזמן אמת ב-ESP32-S3.

- תמיכה בקלט שמע/משתמש נוספים

נכון לעכשיו, כניסת השמע מוגבלת לקבצי wav בכרטיס microSD / גרסאות עתידיות יכולו לשלב מקורות אחרים בנוסף לכך. ניתן ליישם נגן Bluetooth מובנה עדי ליישם הזרמת שמע אלחוטית באמצעות הכלים והיכולות של המיקרו-בקר הנתון לנו. כמו כן, ניתן להסוות את המיקרו-בקר כנגן שמע, אשר כך מזדהה מול המחשב המחובר אליו-דרך כבל usb-type c (בעזרת ספריית tiny-usb) או דרך כל דרך כזאת או אחרת. הן שיפור בקלט שמע והן בקלט המשתמש, יהיה ניתן לשפר את ממשק המשתמש על ידי הוספת יכולות בקרה אלחוטיות, מעבר לחומרה הפיזית. באמצעות מינוף פונקציות ה-Wi-Fi וה-Bluetooth של ה-ESP32 יהיה ניתן לפתח אפליקציה/ווב מבוסס אינטרנט המאורח על הבקר עצמו. זה יאפשר אופן ממשק גרפי אינטואיטיבי יותר, מתוככמים ופוטנציאל להגדרות קבועות מראש ותצורה קלה יותר בהשוואה למערכת הנוכחית.

• סיכום

לסיכום, פרויקט זה הדגים בהצלחה את פיתוחו של אקולייזר גרפי משובץ מחשב עצמאי ונגן שמע wav המשתמש במיקרו-בקר ESP32-S3. על ידי בחירה קפדנית של סביבת הפיתוח ESP-IDF ומינוף פונקציות DSP אופטימליות, התגברנו על האתגר המשמעותי של יישום אקולייזר בעל 5 פסי תדרים בזמן אמת, ותוצאה מכך נוצרה מערכת פונקציונלית שמסוגלת לשנות את השמעת השמע על סמך קלט המשתמש באמצעות

הפוטנציומטרים. הפרויקט משלב ציוד היקפי חומרתי מגוון באמצעות פרוטוקולי תקשורת סטנדרטיים, SPI, I2S, I2C, ADC. בעוד שקיימות מגבלות כגון השמעה מונופונית בלבד וקלט מכרטיס זיכרון, היישום המוצלח מספק בסיס למס' שיפורים פוטנציאליים, כולל כניסות שמע נוספות, שמע סטריאופונית ובקרה אלחוטית שזוהו לפיתוח עתידי. עבודה זו מדגישה את יכולתם של מיקרו-בקרים להתמודד עם משימות עיבוד עתירות חישוב מורכבות בזמן אמת.

נספחים

- פילטרים דיגיטליים, חישוב מקדמי הפילטרים

 peekingFilterGunieaPig.ipynb